পর্ব ৪ · কোডিং · ক্লাস ১৬ / ৩৭

এক ফাইলেই পুরো নিয়ম

সিলেবাস: প্র্যাকটিক্যাল ৫ · টপিক ৫.১ (মডেল ও স্কিমা বাস্তবায়ন)

৩১ মডেল · ২৩২ ফিল্ড · ১৪টি clean() · ৩টি save()

Django model = টেবিল + নিয়ম + আচরণ

শুরু করুন প্রশ্ন দিয়ে

একটি মডেল ফাইলে কি শুধু কলাম থাকে?

হাত তুলুন: শুধু কলাম? না হয় আরও কিছু?

IMS-এর ৩১টি মডেলে ২৩২টি ফিল্ড, ১৪টি clean(), ৩টি save(), ১৪টি @property — সব একই ফাইলে।

কারণ মডেল মানে শুধু টেবিল নয়: **নিয়ম** (কী বৈধ) আর **আচরণ** (এই সারি দিয়ে কী করা যায়) — দুটোই ডেটার সঙ্গেই থাকা উচিত, view-এ ছড়িয়ে নয়।

শিক্ষকের নোট: প্রশ্নটা করুন, তারপর বলুন: "আজ আপনি দেখবেন একই প্রশ্নের উত্তর তিন জায়গায় দেওয়া যায় — ফর্মে, ভিউতে, আর মডেলে। শেষটা কেন সেরা, সেটাই আজকের গল্প।"

ধাপ ১

এক মডেলের তিনটি স্তর

কলাম · নিয়ম · আচরণ — তিনটাই এক ফাইলে, তিনটাই একসাথে যায়

১. কলাম (Fields)

```
name = CharField(max_length=40)
starts_on = DateField()
is_closed = BooleanField()
```

ডেটাবেসে যা দেখবেন

২. নিয়ম (Meta, clean)

```
unique_together = [(workspace, name)]
ordering = ["-starts_on"]
clean(): ends_on > starts_on
```

ভুল ডেটা ঢুকতেই পারে না

৩. আচরণ (Methods)

```
def __str__(self)
@property def is_writable(self)
def save(self, *args, **kwargs)
```

কোডে "লজিক" যেখানে খুঁজে পাবেন

তিনটি স্তর এক ফাইলে থাকলে টেবিলের সাথে তার নিয়ম কখনো বিচ্ছিন্ন হয় না — এটাই Django-র মূল ধারণা।

বিকল্প কি ছিল? নিয়মগুলো ভিউতে `if` দিয়ে লেখা যেত — কিন্তু তখন শেল, অ্যাডমিন, ম্যানেজমেন্ট কমান্ড, ইমপোর্ট — সবাই সেই নিয়ম ভাঙতে পারত।

শিক্ষকের নোট: ছাত্রদের জিজ্ঞেস করুন: "কোন জায়গা থেকে ডেটা আসতে পারে?" (ফর্ম, অ্যাডমিন, শেল, স্ক্রিপ্ট, টেস্ট) — প্রতিটির জন্য আলাদা করে নিয়ম লিখতে হলে একদিনে একটি ভুল হবেই।

ধাপ ২

একটি পূর্ণ মডেল, লাইন ধরে ধরে

`apps/academics/models.py` — AcademicYear, ১৭ লাইন

```
class AcademicYear(TimeStamped):
    workspace = models.ForeignKey(
        Workspace, on_delete=models.CASCADE,
        related_name="academic_years")
    name = models.CharField(max_length=40,
        help_text="e.g. 2026 or 2026-2027")
    starts_on = models.DateField()
    ends_on = models.DateField()
    is_closed = models.BooleanField(default=False)

    class Meta:
        unique_together = [("workspace", "name")]
        ordering = ["-starts_on"]

    def __str__(self):
        return self.name

    def clean(self):
        if self.starts_on and self.ends_on \
                and self.ends_on <= self.starts_on:
            raise ValidationError({"ends_on":
                "End date must be after the start date."})
```

লাইন	সিদ্ধান্ত
`TimeStamped`	created_at · updated_at বিনামূল্যে
workspace FK	স্কোপ কলাম — ক্লাস ১৪
`help_text`	ফর্মেই ব্যাখ্যা দেখাবে
দুটি DateField	তারিখ, সময় নয় — কারণ দিনই দরকার
`unique_together`	এক প্রতিষ্ঠানে এক নাম
`ordering`	নতুন বছর আগে — `-` মানে উল্টো
`__str__`	ড্রপডাউনে নাম দেখাবে
`clean()`	শেষ তারিখ শুরুর পরে

শিক্ষকের নোট: ধীরে ধীরে লাইন ধরে পড়ুন। প্রতিটি লাইনে জিজ্ঞেস করুন: "এটা না থাকলে কী হতো?" — বিশেষ করে `ordering` আর `help_text`, যা ছাত্ররা প্রায়ই বাদ দেয়।

ধাপ ৩

২৩২টি ফিল্ডের ভাঙা হিসাব

৭৫টি সম্পর্ক + ১৫৭টি সাধারণ কলাম — কোনটা কতবার

ধরন	সংখ্যা	কোথায়
`ForeignKey`	৭০	সম্পর্ক
`CharField`	৬২	নাম · শিরোনাম · কোড
`BooleanField`	২৩	is_active · is_published
`DateField`	১৮	তারিখ (দিন)
`DateTimeField`	১৬	created_at · submitted_at
`PositiveIntegerField`	১২	full_marks · seats
`TextField`	৯	বিবরণ · নোট · ফিচার
`DecimalField`	৬	**টাকা** · শতাংশ
বাকি ৬ ধরন	১৪	Email · Slug · IP · File · Image · SmallInt

১৩৯টি ফিল্ড শুধু CharField আর ForeignKey — কারণ বেশিরভাগ ব্যবসা "নাম + সম্পর্ক" দিয়েই চলে।

টাকা সবসময় Decimal — `FloatField` শূন্যটি জায়গায় ভুল হিসাব দেয়।

লক্ষ্য করুন: `FloatField` **একটিও নেই**। সংখ্যা দরকার হলে Integer, টাকা দরকার হলে Decimal — এই দুইয়ের বাইরে ভাবনার দরকার নেই।

শিক্ষকের নোট: Float-এর ফাঁদটা বোর্ডে দেখান: ০.১ + ০.২ = ০.৩০০০০০০০০০০০০০০০৪। বলুন: "টাকার হিসাবে এই ভুল জমে জমে বড় হয়।"

ধাপ ৪

ফিল্ড বাছাই — পাঁচটি সিদ্ধান্ত

প্রতিটি পছন্দের পেছনে একটি প্রশ্ন

প্রশ্ন	বেছে নিন	IMS-এর উদাহরণ
দিন দরকার, নাকি দিন+সময়?	`DateField` / `DateTimeField`	পরীক্ষার তারিখ = Date · জমা দেওয়ার সময় = DateTime
সংখ্যা, নাকি টাকা?	`PositiveIntegerField` / `DecimalField`	পূর্ণ নম্বর ১০০ · ফি ৳১,৫০০.০০
ছোট নাম, নাকি বড় লেখা?	`CharField(max_length)` / `TextField`	শ্রেণির নাম · হোমওয়ার্কের বিবরণ
নির্বাচিত মানগুলোর বাইরে যাবে?	`choices` (TextChoices)	উপস্থিতি: PRESENT · ABSENT · LATE · EXCUSED
ফাইল?	`FileField` / `ImageField`	উপকরণ · প্রতিষ্ঠানের লোগো

max_length বাড়িয়ে দিন যা একদিন বাড়তে পারে — নাম, ব্যাখ্যা।

max_length ঠিক রাখুন যা কখনো বাড়ে না — হেক্স কালার ৭, মুদ্রা ৩, ফোন ১৬।

শিক্ষকের নোট: শেষ দুটি কার্ডের পার্থক্যটা প্রশ্ন করান: "শ্রেণির নাম ৪০ না ২০০?" উত্তর: বাড়তে পারে, তাই বড় রাখুন — কিন্তু TextField নয়, কারণ তালিকায় দেখাতে হবে।

ধাপ ৫

TimeStamped — ১৯টি মডেলের সাধারণ ভিত্তি

`abstract = True` — নিজের টেবিল হয় না, শুধু কলাম ধার দেয়

```
class TimeStamped(models.Model):
    created_at = models.DateTimeField(
        auto_now_add=True)
    updated_at = models.DateTimeField(
        auto_now=True)

    class Meta:
        abstract = True


class AcademicYear(TimeStamped):
    ...
# ১৯টি মডেল এভাবেই শুরু হয়
```

auto_now_add = একবার

তৈরির সময় লেখে, পরে আর বদলায় না — ইতিহাসের সূচনা।

auto_now = প্রতিবার

প্রতিটি `save()`-এ হালনাগাদ — "সর্বশেষ কে ছুঁল?"

abstract কেন

টেবিল না বানিয়ে ১৯ মডেলে কলাম যোগ করে — এক জায়গায় বদলালে সব বদলায়।

যে ১২টি মডেলে এটি নেই, তাদের নিজের সময়-কলাম আছে বা দরকার নেই — যেমন `AuditLog` (`created_at` একটাই), `Plan`, `User` (Django-র নিজস্ব `date_joined`)।

শিক্ষকের নোট: প্রশ্ন করুন: "abstract বাদ দিয়ে কপি-পেস্ট করলে?" উত্তর: ১৯ জায়গায় বদলাতে হবে — আর একদিন কেউ একটি বাদ দেবে।

ধাপ ৬

`__str__` — এক লাইন, বড় পার্থক্য

ড্রপডাউন, অ্যাডমিন, শেল, এরর মেসেজ — সব জায়গায় এটাই দেখায়

```
# Without __str__ — what you see in a dropdown
<AcademicYear: AcademicYear object (3)>
<AcademicYear: AcademicYear object (4)>

# With __str__
2026
2025-2026

# IMS-এর তিনটি ভালো উদাহরণ
def __str__(self):
    return f"{self.number} · {self.workspace.name}
            · {self.amount} {self.currency}"   # Invoice

def __str__(self):
    return f"{self.full_name} - {self.homework.title}"
                                              # Submission

def __str__(self):
    return f"{self.action} by {self.actor_id}"  # AuditLog
```

পরিচয় দেয়

কোন সারি — মানুষ চেনে নামে, ডেটাবেস চেনে id-তে।

ডিবাগ সহজ করে

`print(obj)` করলেই বোঝা যায় কোনটা।

নিয়ম

এক লাইনে, ৮০ অক্ষরের মধ্যে, আর **রহস্যময় তথ্য নয়** (পাসওয়ার্ড, টোকেন কখনো নয়)।

শিক্ষকের নোট: লাইভ দেখান — শেল খুলে একটি মডেল প্রিন্ট করুন যেখানে `__str__` নেই। ওই `<... object (3)>` দেখেই ছাত্ররা বুঝবে কেন এটা দরকার।

ধাপ ৭

clean() বনাম constraint — কে কোথায় থামায়

১৪টি `clean()` (পাইথন) · ২টি `CheckConstraint` (ডেটাবেস)

	clean() · ফর্ম-স্তর	CheckConstraint · ডেটাবেস
কোথায় চলে	`full_clean()`, ModelForm সেভের আগে	প্রতিটি INSERT/UPDATE-এ
ভালো কী করে	বন্ধুসুলভ বার্তা, কোন ফিল্ডে ভুল	কোনো পথেই ভুল ঢুকতে দেয় না
কে এড়াতে পারে	`save()` সরাসরি ডাকলে	কেউ না — এমনকি শেলও না
IMS-এ উদাহরণ	শেষ তারিখ শুরুর পরে · অন্য প্রতিষ্ঠানের ছাত্র নয়	`grade_band_min_le_max` · `coordinator_scope_needs_a_target`

দুটোই রাখুন: clean দেয় ভালো এরর, constraint দেয় শেষ রক্ষা।

শুধু clean যথেষ্ট নয় — `Model.objects.create()` clean ডাকে না, সোজা সেভ করে।

শিক্ষকের নোট: দ্বিতীয় কার্ডটাই আসল ফাঁদ। বলুন: "আপনার ভ্যালিডেশন কাজ করছে — কারণ আপনি ফর্ম দিয়ে টেস্ট করেছেন। শেল দিয়ে টেস্ট করুন।"

ধাপ ৮

@property — ব্যবসার নিয়ম ডেটার ঘরে

১৪টি property; সেরাটি `Subscription.is_writable`

```
@property
def ends_on(self):
    return (self.trial_ends_on
            if self.status == self.Status.TRIAL
            else self.current_period_end)

@property
def days_left(self):
    end = self.ends_on
    return None if end is None \
        else (end - timezone.localdate()).days

@property
def is_writable(self) -> bool:
    """TRIAL আর ACTIVE পারেন। PAST_DUE পড়তে
    পারে, লিখতে পারে না। ডেটা কখনো মোছে না।"""
    if self.status in (self.Status.PAST_DUE,
                       self.Status.SUSPENDED,
                       self.Status.CANCELLED):
        return False
    if self.status == self.Status.TRIAL:
        return self.trial_ends_on is None \
            or self.trial_ends_on >= timezone.localdate()
    return True
```

তিন জায়গায় এক নিয়ম

মিডলওয়্যার, ব্যানার, রিপোর্ট — সবাই `is_writable` পড়ে। কোথাও আলাদা করে লেখা নেই।

নিয়ম বদলালে এক জায়গায়

"ট্রায়াল শেষ = read-only" — এই সিদ্ধান্ত একটাই ফাংশনে।

টেস্ট সহজ

টেস্টে আলাদা রেকর্ড না বানিয়ে শুধু তারিখ বদলে property যাচাই করা যায়।

শিক্ষকের নোট: প্রশ্ন করুন: "এই নিয়মটা তিন জায়গায় কপি করলে?" উত্তর: তিন মাস পরে এক জায়গায় বদলাবেন, দুই জায়গায় ভুল থেকে যাবে — আর ব্যবহারকারী বলবে "কখনো কাজ করে, কখনো করে না।"

ধাপ ৯

save() — মাত্র তিনটি জায়গায়, আর সবগুলোই হিসাব

Enrollment · Submission · Invoice — তিনটিই এমন কিছু গণনা করে যা কেউ ভুলে যেতে পারে

```python
# ১. Invoice — নম্বর কখনো ফাঁকা থাকে না
def save(self, *args, **kwargs):
    if not self.number:
        self.number = self._next_number()
    super().save(*args, **kwargs)

@staticmethod
def _next_number() -> str:
    year = timezone.localdate().year
    prefix = f"{settings.INVOICE_PREFIX}-{year}-"
    last = (Invoice.objects
            .filter(number__startswith=prefix)
            .order_by("-number")
            .values_list("number", flat=True).first())
    n = int(last.split("-")[-1]) + 1 if last else 1
    return f"{prefix}{n:04d}"

# ২. Submission — দেরিতে জমা স্বয়ংক্রিয়
def save(self, *args, **kwargs):
    if self.homework_id and not self.pk:
        self.is_late = timezone.now() > self.homework.due_at
    super().save(*args, **kwargs)
```

নম্বর কেন এখানে

ইনভয়েস কোথা থেকেই তৈরি হোক — নম্বর থাকবেই। `IMS-2026-0001`

কেন `not self.pk`

শুধু প্রথম সেভে — পরে সম্পাদনা করলে `is_late` বদলাবে না।

Enrollment — তৃতীয়টি

`is_active=False` হলে `ended_on` নিজেই বসে যায়।

ফাঁদ: দুই অনুরোধ একসাথে এলে একই নম্বর আসতে পারে — তাই unique + চাপে `Invoice.number`।

শিক্ষকের নোট: সততার সাথে বলুন: "এই নম্বর-বানানোর পদ্ধতি সাধারণ চাপে ঠিক, ভিড়ে দুর্বল। বড় সিস্টেমে ডেটাবেস sequence লাগে।" — সীমাবদ্ধতা জানাই ভালো engineering.

ধাপ ১০

Manager — ব্যবহারকারী তৈরির একমাত্র দরজা

`apps/accounts/managers.py` — কেউ যেন প্লেইন পাসওয়ার্ড লিখতে না পারে

```
class UserManager(BaseUserManager):
    use_in_migrations = True

    def _create_user(self, email, password, **extra):
        if not email:
            raise ValueError("An e-mail address is required.")
        email = self.normalize_email(email).lower()
        user = self.model(email=email, **extra)
        user.set_password(password)   # hashes, never stores plain
        user.save(using=self._db)
        return user

    def create_user(self, email, password=None, **extra):
        extra.setdefault("is_staff", False)
        extra.setdefault("is_superuser", False)
        return self._create_user(email, password, **extra)

    def create_superuser(...):
        ... is_staff=True, is_superuser=True
        return self._create_user(...)
```

ইমেইল নরমালাইজ

সব ছোট হাতে — `Admin@IMS.local` আর `admin@ims.local` একই মানুষ।

set_password

প্লেইন পাসওয়ার্ড কখনো ডেটাবেসে যায় না — হ্যাশ যায়।

কেন ম্যানেজার

`User(email=..., password="abcd")` লিখলে প্লেইন পাসওয়ার্ড সেভ হয়ে যেত — এই ভুলটা কেউ করতেই পারবে না।

শিক্ষকের নোট: বলুন: "ভালো API মানে ভুল কাজটা কঠিন করে তোলা। এখানে ভুলটা প্রায় অসম্ভব — কারণ সব দরজা এক।"

ধাপ ১১

choices — ১১টি জায়গায়, খালি স্ট্রিং নয়

`models.TextChoices`: মান আর লেবেল একসাথে

```
class AttendanceRecord(TimeStamped):
    class Status(models.TextChoices):
        PRESENT = "PRESENT", "Present"
        ABSENT = "ABSENT", "Absent"
        LATE = "LATE", "Late"
        LEAVE = "LEAVE", "Leave"

    status = models.CharField(
        max_length=10, choices=Status.choices,
        default=Status.PRESENT)

# কোডে:
record.status == AttendanceRecord.Status.LATE
record.get_status_display()   # → "Late"
```

কোথায়	কী বাছাই
উপস্থিতি	PRESENT · ABSENT · LATE · LEAVE
ভূমিকা	ORG_ADMIN · COORDINATOR · TEACHER · STUDENT
প্রতিষ্ঠানের ধরন	INSTITUTION · PERSONAL
সাবস্ক্রিপশন	TRIAL · ACTIVE · PAST_DUE · SUSPENDED · CANCELLED
পেমেন্ট	bKash · Nagad · Bank · Cash · Other
হোমওয়ার্ক জমা	SUBMITTED · REVIEWED · RETURNED

লেবেল বদলালে ইংরেজি লেখাটা বদলায়, মানটি থাকে — তাই পুরোনো ডেটা ভাঙে না।

ফাঁদ: `"present"` আর `"PRESENT"` — ছোট-বড় হাতের একটি অক্ষরে রিপোর্ট ভুল দেখাবে, আর এররও আসবে না।

শিক্ষকের নোট: ছাত্রদের জিজ্ঞেস করুন: "choices না দিলে কী হারাবেন?" উত্তর: ড্রপডাউন, লেবেল, আর ভুল মান আটকানোর ক্ষমতা।

অনুশীলন

২২ মিনিট — নিজের প্রথম তিনটি মডেল

কপি নয়, ক্রম মানুন

১. TimeStamped লিখুন (৩ মিনিট)

abstract base, দুইটি সময়-কলাম। এরপর সব মডেল এটি থেকে শুরু হবে।

২. একটি মূল মডেল (৭ মিনিট)

স্কোপ FK + নাম + ২টি তারিখ/সংখ্যা + `Meta`-তে unique ও ordering + `__str__`।

৩. নিয়ম যোগ করুন (৭ মিনিট)

একটি `clean()` (তারিখের ক্রম) আর একটি `choices`। দুই জায়গায় ভুল ডেটা ঢোকার পথ বন্ধ করুন।

৪. যাচাই (৫ মিনিট)

শেল খুলুন: `full_clean()` দিয়ে ভুল ডেটা দিন — এরর আসছে? তারপর `objects.create()` দিয়ে একই ভুল দিন — কী হলো?

চতুর্থ ধাপেই আজকের সবচেয়ে বড় শিক্ষা: **clean() শুধু ফর্ম পাহারায়, constraint ডেটাবেস পাহারায়**।

শিক্ষকের নোট: প্রত্যেকে যেন চতুর্থ ধাপটা নিজের হাতে করে — ওই দুই আউটপুট পাশাপাশি দেখলেই আর কোনো ব্যাখ্যা লাগবে না।

আলোচনা

ক্লাসে আলোচনার চার প্রশ্ন

উত্তর এক বাক্যে

- নিয়ম ফর্মে লিখলে কী কী পথ খোলা থাকে?
- `clean()` আর `CheckConstraint` — দুটোই রাখবেন কেন?
- `@property` বনাম ভিউতে হিসাব — কোনটা কোথায় রাখা উচিত?
- `save()` override-এ ব্যবসার নিয়ম লেখা কখন বিপজ্জনক?

শিক্ষকের নোট: শেষ প্রশ্নে বলুন: "bulk_create, update(), loaddata — এগুলো save() ডাকে না। তাই খুব জরুরি নিয়ম save()-এ নয়, constraint-এ রাখা ভালো।"

সারমর্ম

ছয়টি কথা সঙ্গে নিন

প্রতিটির পাশে IMS-এর প্রমাণ

নীতি	প্রমাণ
মডেল = কলাম + নিয়ম + আচরণ	৩১ মডেল · ২৩২ ফিল্ড · ১৪ clean · ১৪ property
সাধারণ কলাম abstract base-এ	`TimeStamped` — ১৯টি মডেল
টাকায় Decimal, কখনো Float নয়	DecimalField ৬টি · FloatField ০টি
নিয়ম ডেটাবেসেও রাখুন	২টি CheckConstraint + ৮টি UniqueConstraint
ব্যবসার নিয়ম ডেটার ঘরে	`Subscription.is_writable` — মিডলওয়্যার, ব্যানার, রিপোর্ট সবাই পড়ে
ভুল কাজ কঠিন করে তুলুন	UserManager — প্লেইন পাসওয়ার্ড সেভ করা অসম্ভব

শিক্ষকের নোট: পঞ্চম সারিটা আবার বলুন: "নিয়ম একটা জায়গায়, ব্যবহার বহু জায়গায়।" এটাই ভিউ-এ লজিক না রাখার আসল কারণ।

১৭

পরের ক্লাস — "অনুরোধ কোথায় যায়"

ভিউ ও URL: ৮৫টি রুট · function-based view · decorator · 404/403 · pagination

পর্ব ৪ · কোডিং · ক্লাস ১৭ / ৩৭

অনুরোধ কোথায় যায়

সিলেবাস: প্র্যাকটিক্যাল ৫ · টপিক ৫.২ (ভিউ ও রাউটিং)

৮৫টি রুট · ৫৭টি স্কোপড ভিউ · ৪টি গন্তব্য: render · redirect · 404 · 403

URL → decorator → view → template

শুরু করুন প্রশ্ন দিয়ে

একটি ক্লিকের পরে কতটা সিদ্ধান্ত নেওয়া হয়?

হাত তুলুন: একটি ভিউতে ক'টি সিদ্ধান্ত? ৩? ৫? ১০?

৮৫টি রুট, কিন্তু ৫৭টি ভিউ একই দরজা দিয়ে ঢোকে — @workspace_view, যে চারটি প্রশ্নের উত্তর আগেই দিয়ে দেয়।

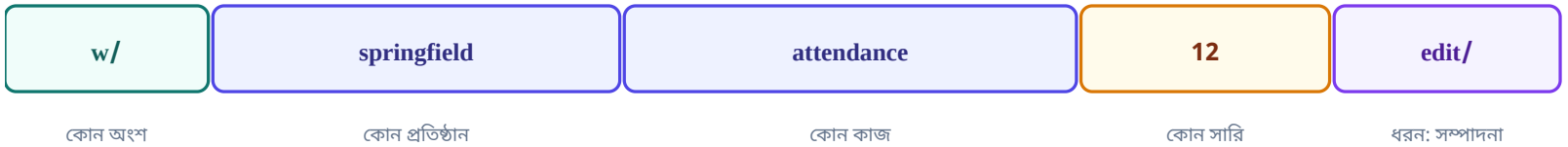
আজকের ক্লাসে প্রতিটি ভিউর ভেতরের ক্রমটা দেখব: কে ঢুকতে পারবে (৪০৪), ভূমিকা কী (৪০৩), কী ডেটা (স্কোপ), তারপর সেভ নাকি দেখানো (POST/GET), শেষে কোথায় ফেরত (redirect)।

শিক্ষকের নোট: বলুন: "ভিউ হলো দরজার প্রহরী — কিন্তু প্রহরীর কাজ বন্ধুকে চেনা নয়, অচেনাকে ঠেকানো।" এই বাক্যেই ক্লাসের সুর।

ধাপ ১

একটি URL পড়তে শেখা

`w/springfield/attendance/12/edit/` — প্রতিটি অংশের মানে আছে



প্যাটার্ন: w/<slug:slug>/attendance/<int:pk>/edit/

৫৭টি ভিউর URL-ই প্রথমে প্রতিষ্ঠানের slug নেয় — তাই টেন্যান্ট ঠিকানা থেকে আসে, সেশনের অনুমানে নয়।

ব্রাউজারে দেখা যায়, শেয়ার করা যায়, বুকমার্কে থাকে — আর ভুল ঠিকানায় গেলে ৪০৪, চুরি করতে গেলে ৪০৩।

বিকল্প: প্রতিষ্ঠান সেশন থেকে নেওয়া (slug ছাড়া URL)। কিন্তু তাহলে দুই ট্যাবে দুই প্রতিষ্ঠান খোলা যায় না — আর শেয়ার করা লিংক অন্য প্রতিষ্ঠানে গিয়ে খোলে। তাই slug URL-এ।

শিক্ষকের নোট: ছাত্রদের বলুন নিজের প্রজেক্টের একটি URL লিখতে, তারপর প্রতিটি অংশে প্রশ্ন করুন "এটা কেন এখানে?" — যে অংশের উত্তর নেই, সেটা বাদ দেওয়ার চেষ্টা করুন।

ধাপ ২

URLconf — ১১টি অ্যাপ, ১১টি include

প্রতিটি অ্যাপের নিজের `urls.py`, নিজের `app_name`: মোট ৮৫টি নাম

```
# config/urls.py
urlpatterns = [
    path("admin/", admin.site.urls),
    path("", include("apps.accounts.urls")),
    path("", include("apps.workspaces.urls")),
    path("", include("apps.core.urls")),
    path("", include("apps.academics.urls")),
    path("", include("apps.students.urls")),
    path("", include("apps.attendance.urls")),
    path("", include("apps.lessons.urls")),
    path("", include("apps.assessments.urls")),
    path("", include("apps.reports.urls")),
    path("", include("apps.audit.urls")),
    path("", include("apps.billing.urls")),
]

# apps/attendance/urls.py
app_name = "attendance"
urlpatterns = [
    path("w/<slug:slug>/attendance/", views.take, name="take"),
    path("w/<slug:slug>/attendance/register/", views.register,
name="register"),
    path("w/<slug:slug>/attendance/summary/", views.summary,
name="summary"),
    path("w/<slug:slug>/attendance/<int:pk>/edit/",
         views.record_edit, name="record_edit"),
```

app_name কেন

`"attendance:take"` — একই নাম দুই অ্যাপে থাকতে পারে, সংঘর্ষ নেই।

কোডে নাম, URL নয়

`redirect("attendance:register", slug=slug)` — ঠিকানা বদলালে এক জায়গায় বদলায়।

ক্রম মানে

উপরের প্যাটার্ন আগে মেলে — নির্দিষ্ট পথ সাধারণ পথের আগে রাখুন।

শিক্ষকের নোট: বলুন: "কোথাও যদি `"/w/x/attendance/"` হার্ডকোড করা থাকে, সেটাই লুকানো বাগের জন্ম।" নিজের প্রজেক্টে grep করতে বলুন।

ধাপ ৩

একটি ভিউ, লাইন ধরে ধরে

`apps/attendance/views.py` — `take()`, উপস্থিতি নেওয়ার পর্দা

```python
@workspace_view(*TEACHING)
def take(request, slug):
    ws = request.workspace                      # decorator দিয়েছে
    classes = visible_classes(request.user, ws,
                              request.membership)   # স্কোপ
    class_id = request.GET.get("class") or request.POST.get("class_group")
    form = AttendanceBulkForm(request.POST or None,
                              workspace=ws, user=request.user,
                              class_id=class_id)
    cls = None
    if class_id:
        cls = classes.filter(pk=class_id).first()
        if cls is None:
            raise PermissionDenied("You are not assigned to that class.")
        if request.method == "POST" and form.is_valid():
            day = form.cleaned_data["date"]
            if day > timezone.localdate():
                messages.error(request, "Attendance cannot be taken for a future date.")
            else:
                created, updated = _save_day(request, cls, ...)
                log(request, "attendance.save", cls, summary=...)
                messages.success(request, f"Attendance saved: {created} new, ...")
                return redirect(f"{request.path}?class={cls.pk}&date={day}")
    return render(request, "attendance/take.html", {...})
```

ধাপ	কী হচ্ছে
decorator	লগইন + প্রতিষ্ঠান + ভূমিকা
`visible_classes`	শিক্ষক শুধু নিজের শ্রেণি
`classes.filter(pk=…)`	অন্যের শ্রেণি চাইলে **৪০৩**
`request.POST or None`	GET-এ ফর্ম খালি, POST-এ ভরা
`form.is_valid()`	নিয়ম ফর্মে, ভিউতে নয়
`_save_day()`	এক transaction-এ সব ছাত্র
`log(...)`	অডিট — কে কখন
`redirect` / `render`	সেভের পরে সরাসরি HTML নয়

শিক্ষকের নোট: বলুন: "লক্ষ্য করুন ভিউতে একটি নিয়মও লেখা নেই — কোন ফিল্ড বাধ্যতামূলক, কে অনুমোদিত, সব অন্য জায়গায়। ভিউ শুধু সাজায়।"

ধাপ ৪

@workspace_view — এক decorator, চার সিদ্ধান্ত

`apps/core/permissions.py` · ৫৭টি ভিউতে ব্যবহৃত

```python
def workspace_view(*allowed_roles, allow_platform_admin=True):
    def decorator(view_func):
        @wraps(view_func)
        def _wrapped(request, slug, *args, **kwargs):
            if not request.user.is_authenticated:
                return redirect(f"/accounts/login/?next={request.path}")

            workspace = Workspace.objects.filter(
                slug=slug, is_active=True).first()
            if workspace is None:
                raise Http404("Workspace not found.")

            membership = get_membership(request.user, workspace)
            if membership is None:
                # প্ল্যাটফর্ম অ্যাডমিনও অন্যের ব্যক্তিগত
                # রেকর্ড দেখতে পারে না
                raise Http404("Workspace not found.")

            if allowed_roles and membership.role not in allowed_roles:
                raise PermissionDenied(
                    "Your role in this workspace does not allow this action.")

            request.workspace = workspace
            request.membership = membership
            request.session["active_workspace_id"] = workspace.pk
            return view_func(request, slug, *args, **kwargs)
        return _wrapped
    return decorator
```

১. লগইন নেই → redirect

`?next=` সহ — লগইনের পরে ফিরে আসবে।

২. প্রতিষ্ঠান নেই → ৪০৪

অস্তিত্বই স্বীকার করি না।

৩. সদস্য নয় → ৪০৪

"আছে কিন্তু নিষেধ" বললে প্রতিষ্ঠানের অস্তিত্ব ফাঁস হয়।

৪. ভূমিকা মেলে না → ৪০৩

এখন সে ভেতরে, তাই সত্য বলা যায়।

+ উপহার

`request.workspace` আর `request.membership` — ভিউতে আর খুঁজতে হয় না।

শিক্ষকের নোট: ৩ ও ৪-এর পার্থক্যটাই ক্লাসের প্রাণ। জিজ্ঞেস করুন: "অচেনা মানুষকে কী বলবেন — 'ঘর আছে কিন্তু ঢুকতে দেব না', নাকি 'এমন ঘর নেই'?"

ধাপ ৫

৪০৩ নাকি ৪০৪ — একটি সিদ্ধান্ত, বড় পার্থক্য

কারণ: সদস্যতা জানা মানেই প্রতিষ্ঠানের অস্তিত্ব জানা

পরিস্থিতি	কোড	কেন এটাই
লগইন নেই	৩০২ → login	`?next=` দিয়ে ফেরত — ব্যবহারকারী বিভ্রান্ত নয়
slug নেই / প্রতিষ্ঠান বন্ধ	**৪০৪**	ঠিকানা ভুল — পর্দাই নেই
অন্য প্রতিষ্ঠানের slug	**৪০৪**	৪০৩ বললে "এই প্রতিষ্ঠান আছে" ফাঁস হতো
সদস্য, কিন্তু ভূমিকা নয়	**৪০৩**	সে ভেতরের মানুষ — লুকানোর কিছু নেই
শিক্ষক, অন্য শ্রেণির আইডি	**৪০৩**	স্কোপের বাইরে, কিন্তু প্রতিষ্ঠান তার নিজের
ছাত্র, ক্লাসের ফল এক্সপোর্ট	**৪০৩**	"Students cannot export class results."

ভালো এরর: কী হয়নি + কেন + এখন কী করব (ক্লাস ১২-এর বার্তা-নিয়ম)।

খারাপ: সব জায়গায় ৪০৪ — ব্যবহারকারী ভাবেন সাইট নষ্ট, সাপোর্টে ফোন করেন।

শিক্ষকের নোট: বলুন: "৪০৩ আর ৪০৪-এর পার্থক্য আক্রমণকারীর জন্য ছোট, ব্যবহারকারীর জন্য বিশাল। তাই সদস্যের জন্য ৪০৩, অচেনার জন্য ৪০৪।"

ধাপ ৬

get_object_or_404 — ৩৫ জায়গায়, সবসময় স্কোপসহ

আইডি দিয়ে খোঁজা মানে এখনো টেন্যান্ট যাচাই করা হয়নি

```
# বিপজ্জনক — অন্য প্রতিষ্ঠানের সারিও আসে
rec = AttendanceRecord.objects.get(pk=pk)

# IMS-এর নিয়ম — স্কোপ ভিউতেই
rec = get_object_or_404(
    AttendanceRecord, pk=pk,
    workspace=request.workspace)

# স্কোপের বাইরে গেলে আলাদা ৪০৩
if not teacher_can_access_class(request.user, rec.class_group):
    raise PermissionDenied(
        "You are not assigned to this class.")
```

দুই স্তরের চেক

আগে প্রতিষ্ঠান (৪০৪), পরে শ্রেণি-স্কোপ (৪০৩) — দুই প্রশ্ন, দুই উত্তর।

কেন `.get()` নয়

`.get()` ব্যর্থ হলে ৫০০ — ব্যবহারকারী ভাবেন সার্ভার ভাঙা।

সংখ্যায়

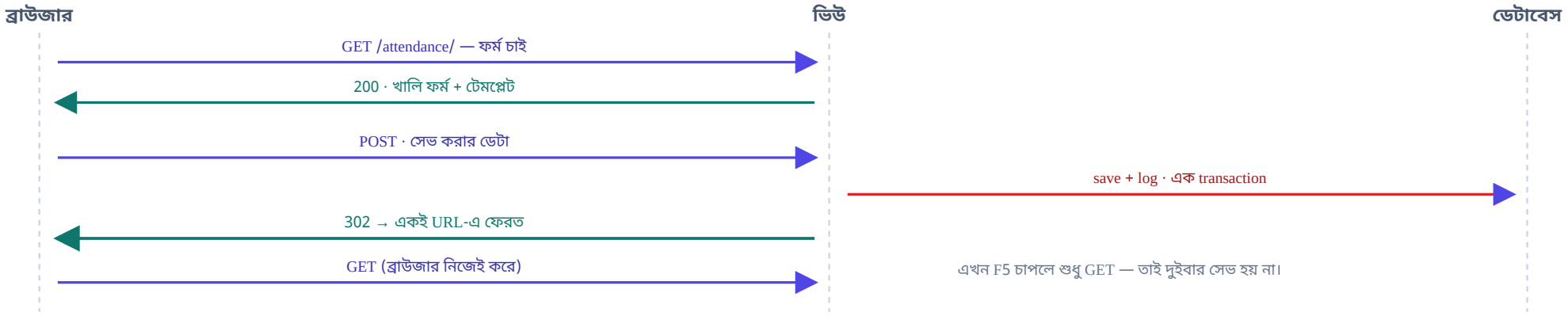
৩৫টি `get_object_or_404` · ২৭টি `PermissionDenied` — কোডবেস জুড়ে একই প্যাটার্ন।

শিক্ষকের নোট: প্রথম লাইনটা বোর্ডে লিখুন, পাশে "কোন প্রতিষ্ঠানের?" প্রশ্ন দিন। উত্তর না থাকলে বলুন: "এই একটা লাইনই পুরো সিস্টেম ফাঁস করতে পারে।"

ধাপ ৭

এক ভিউ, দুই কাজ — GET দেখায়, POST বদলায়

সেভের পরে সবসময় redirect (POST → Redirect → GET) · ৬৬টি redirect



কেন redirect: নাহলে রিফ্রেশে ফর্ম আবার সাবমিট হয় — দুইবার উপস্থিতি।

কেন `request.POST or None`: একটি ভিউতেই দেখানো ও সেভ — দুই ভিউ নয়।

শিক্ষকের নোট: লাইভে F5 চাপুন সেভের পরে — যদি "আবার জমা দিবেন?" বার্তা আসে, বোঝা যায় redirect বাদ পড়েছে। এটা ছাত্রদের সবচেয়ে সাধারণ ভুল।

ধাপ ৮

তালিকা মানেই pagination

দুই জায়গায় ব্যবহৃত: শিক্ষার্থী তালিকা ২৫/পাতা · অডিট লগ ৪০/পাতা

```
qs = (_visible_students(request)
      .select_related("account"))
# সার্চ · ফিল্টার
if q:
    qs = qs.filter(full_name__icontains=q) \
           | qs.filter(admission_no__icontains=q)
if status:
    qs = qs.filter(status=status)
qs = qs.distinct().order_by("full_name")

from django.core.paginator import Paginator

page = Paginator(qs, 25).get_page(request.GET.get("page"))
return render(request, "students/list.html", {
    "page_obj": page,
    "q": q, "status": status,
})
```

get_page, page নয়

`page=999` এলেও ক্র্যাশ করে না — শেষ পাতা দেয়।

ফিল্টার আগে, pagination পরে

ক্রম উল্টালে "২৫টি ফল" ভুল সংখ্যা দেখাবে।

কেন ২৫ আর ৪০

শিক্ষার্থীর সারি লম্বা, অডিটের সারি ছোট — পর্দার মাপ অনুযায়ী।

ফাঁদ: `|` ইউনিয়ন

দুই কোয়েরি জোড়া দিলে `distinct()` লাগে — আর বড় টেবিলে ধীর। `Q(...)` ভালো।

শিক্ষকের নোট: শেষ কার্ডটা সততার সাথে বলুন: "এই কোডটা কাজ করে, কিন্তু এটাই সেরা নয় — `Q(full_name__icontains=q) | Q(admission_no__icontains=q)` লিখলে একটি কোয়েরি, distinct-ও লাগে না।"

ধাপ ৯

কুয়েরি সংখ্যা — ভিউর আসল খরচ

একটি তালিকা পর্দা ৩টি কুয়েরিতে হতে পারে, ৩০৩টিতেও

```
# N+1: প্রতিটি সারির জন্য আলাদা কুয়েরি
for e in cls.enrollments.all():
    print(e.student.full_name)
#   1 + N কুয়েরি

# IMS: join আগেই
for e in cls.enrollments.select_related(
        "student").order_by("student__full_name"): ...
#   1 কুয়েরি

# গণনা ডেটাবেসে, পাইথনে নয়
agg = qs.values("student__full_name").annotate(
    total=Count("id"),
    present=Count("id", filter=Q(status__in=["PRESENT", "LATE"])),
).order_by("student__full_name")
```

কৌশল	কখন
`select_related`	FK/1:1 — JOIN করে আনে
`prefetch_related`	M:N বা উল্টো দিকের তালিকা
`values()`	শুধু দরকারি কলাম — মডেল বানায় না
`annotate(Count)`	পাইথনে লুপ নয়, ডেটাবেসে গণনা
`exists()` / `count()`	পুরো সারি টেনে না এনে প্রশ্নের উত্তর

মাপার উপায়: `connection.queries` বা Django Debug Toolbar — অনুমান নয়।

শিক্ষকের নোট: বলুন: "৫০ জন ছাত্রের ক্লাসে ৫১টি কুয়েরি = ৫১ বার নেটওয়ার্ক। ৫০০ জনে সেটাই সেকেন্ডে পরিণত হয়।" সংখ্যাটা বোর্ডে লিখুন।

সতর্কতা

ভিউ-এ পাঁচটি সাধারণ ভুল

প্রতিটির প্রতিকার একটি লাইনে

ভুল	পরিণতি ও প্রতিকার
১. আইডি দিয়ে সরাসরি `.get(pk=…)`	অন্য প্রতিষ্ঠানের ডেটা দেখা যায় → `workspace=request.workspace` যোগ করুন
২. সেভের পরে `render`	রিফ্রেশে দুইবার সেভ → `redirect` (PRG)
৩. সব ভুলে ৪০৪	ভূমিকা ভুল হলেও ৪০৪ → সদস্যের জন্য ৪০৩
৪. ভিউতে ব্যবসার নিয়ম	দুই ভিউ দুই রকম হিসাব → নিয়ম মডেল/সার্ভিসে
৫. তালিকায় pagination নেই	৫০০০ সারির পর্দা ধীরে মরে → `Paginator`

একটাই প্রশ্ন সবগুলো ধরে: **"এই ভিউটি কোন প্রতিষ্ঠানের?"** — উত্তর কোডে লেখা থাকলে প্রথম ভুলটাও থাকবে না।

শিক্ষকের নোট: প্রতিটি ভুলের জন্য একজন ছাত্রকে নিজের প্রজেক্টে খুঁজতে বলুন — ৫ মিনিটে ৫টি বাস্তব উদাহরণ পাবেন, আর সেগুলোই স্মৃতিতে থাকবে।

অনুশীলন

২০ মিনিট — নিজের তিনটি রুট

তালিকা · তৈরি · সম্পাদনা

১. URL লিখুন (৫ মিনিট)

তিনটি রুট, প্রতিটিতে স্কোপ: `/w/<slug>/items/` · `/new/` · `/<int:pk>/edit/`। `app_name` দিন।

২. decorator লিখুন (৭ মিনিট)

লগইন নেই → redirect · সদস্য নয় → ৪০৪ · ভূমিকা নয় → ৪০৩ · সফল হলে `request.workspace` বসান।

৩. ভিউ তিনটি (৫ মিনিট)

তালিকায় Paginator + search; তৈরি ও সম্পাদনায় `request.POST or None` + সেভের পরে redirect।

৪. যাচাই (৩ মিনিট)

অন্য একটি প্রতিষ্ঠান/অন্য ভূমিকা দিয়ে লগইন করে দেখুন — ৪০৪ আর ৪০৩ ঠিক জায়গায় আসছে?

বাড়তি কাজ: ব্যবহারকারীকে লগআউট করে সরাসরি সম্পাদনার লিংক দিন — লগইনের পরে সে ওই পাতায় ফিরে আসছে? (`?next=`)

শিক্ষকের নোট: চতুর্থ ধাপে সবাই যেন দুটি ভিন্ন ভূমিকা দিয়ে টেস্ট করে — এটাই আসল অভ্যাস, আর পরের পর্বে টেস্ট লেখার ভিত্তি।

আলোচনা

ক্লাসে আলোচনার চার প্রশ্ন

উত্তর এক বাক্যে

- slug URL-এ না রেখে সেশন থেকে নিলে কী কী ভেঙে যেত?
- সদস্য নয় এমন ব্যবহারকারীকে ৪০৪ কেন, ৪০৩ নয়?
- সেভের পরে `render` করলে ব্যবহারকারী কী ভুল দেখবেন?
- একটি ভিউতে ব্যবসার নিয়ম লিখলে ভবিষ্যতে কী হয়?

শিক্ষকের নোট: তৃতীয় প্রশ্নের উত্তর একজন ছাত্রকে লাইভ দেখাতে দিন (F5 চেপে ডুপ্লিকেট সাবমিট) — মুখে শোনার চেয়ে দেখলে অনেক ভালো মনে থাকে।

সারমর্ম

ছয়টি কথা সঙ্গে নিন

প্রতিটির পাশে IMS-এর প্রমাণ

নীতি	প্রমাণ
প্রতিষ্ঠান URL-এ, সেশনে নয়	৫৭টি রুট `w/<slug>/` দিয়ে শুরু
এক decorator, চার সিদ্ধান্ত	`@workspace_view` ৫৭ ভিউতে
অচেনা → ৪০৪, সদস্য → ৪০৩	`permissions.py`-এর দুই শাখা
আইডি দিয়ে খুঁজলেও স্কোপ লাগে	৩৫টি `get_object_or_404(..., workspace=…)`
সেভের পরে redirect	৬৬টি redirect · ৬৫টি messages
তালিকা মানেই pagination ও select_related	২৫/৪০ প্রতি পাতা · JOIN দিয়ে N+1 এড়ানো

শিক্ষকের নোট: দ্বিতীয় ও চতুর্থ সারি একসাথে বলুন: "প্রহরী দরজায়, আর প্রতিটি দরজার ভেতরেও একবার চেক।" — defense in depth-এর প্রথম পাঠ।

১৮

পরের ক্লাস — "ভুল ইনপুট, ভদ্র উত্তর"

ফর্ম ও ভ্যালিডেশন: ModelForm · field.html · এরর দেখানো · ফাইল আপলোডের নিয়ম

পর্ব ৪ · কোডিং · ক্লাস ১৮ / ৩৭

ভুল ইনপুট, ভদ্র উত্তর

সিলেবাস: প্র্যাকটিক্যাল ৫ · টপিক ৫.৩ (ফর্ম ও ভ্যালিডেশন)

৩৫টি ফর্ম · ১০টি clean · ১২টি widget · ৪টি পাসওয়ার্ড নিয়ম

ModelForm · fields whitelist · add_error

শুরু করুন প্রশ্ন দিয়ে

ভুল ডেটা কোথায় আটকানো উচিত?

হাত তুলুন: ব্রাউজারে? ভিউতে? ফর্মে? ডেটাবেসে?

IMS-এ তিন জায়গায় আটকায় — ফর্ম (ভদ্র বার্তা), মডেল (নিয়ম), ডেটাবেস (শেষ রক্ষা)। আজ প্রথমটির কারিগরি।

ফর্মের কাজ শুধু "ডেটা নেওয়া" নয় — **সেই ডেটাকে বিশ্বাসযোগ্য বানানো**। ৩৫টি ফর্মে ১০টি জায়গায় নিজস্ব নিয়ম লেখা আছে, আর সেগুলোই ব্যবহারকারীর দেখা প্রথম এরর।

শিক্ষকের নোট: উত্তর সাধারণত "ব্রাউজারে" আসে। বলুন: "ব্রাউজারের ভ্যালিডেশন বন্ধ করা যায় একটি বাটনে — তাই সেটা ভদ্রতা, নিরাপত্তা নয়।"

ধাপ ১

ফর্মের চারটি দায়িত্ব

এক ক্লাস, চার কাজ — আলাদা করে ভাবলে কোড পরিষ্কার থাকে

১. সংগ্রহ

POST → cleaned_data

প্রতিটি ফিল্ডের ধরন

"12" → 12 (int)

স্ট্রিং নয়, সত্যিকারের মান

২. যাচাই

required · min_length

clean_<field>()

clean() — দুই ফিল্ডের সম্পর্ক

নিয়ম একবার, সব পথে

৩. ব্যাখ্যা

errors[field]

non_field_errors

help_text

কোন ঘর, কেন, কী করব

৪. সংরক্ষণ

form.save(commit=False)

workspace বসানো

তারপর save()

স্কোপ ভিউ থেকে আসে

চারটি কাজ একই ক্লাসে থাকলে ভিউ ছোট থাকে — আর ব্যবহারকারী সব পর্দায় একই ধরনের বার্তা পান।

শিক্ষকের নোট: বলুন: "ভিউ যদি ফর্ম আর টেমপ্লেটের মাঝের মধ্যস্থতাকারী হয়, তাহলে তার কাজ কম — এটাই ভালো ডিজাইন।"

ধাপ ২

ModelForm — `fields` হলো হোয়াইটলিস্ট

কোন কলাম ফর্মে আসবে, সেটাই নিরাপত্তার প্রথম সিদ্ধান্ত

```
class StudentForm(BootstrapFormMixin, forms.ModelForm):
    class Meta:
        model = StudentProfile
        fields = ["admission_no", "full_name",
                  "guardian_name", "guardian_phone",
                  "date_of_birth", "notes"]
        widgets = {"date_of_birth": forms.DateInput(
            attrs={"type": "date"})}

    def __init__(self, *args, workspace=None, **kwargs):
        super().__init__(*args, **kwargs)
        self.workspace = workspace

# ভিউতে:
obj = form.save(commit=False)
obj.workspace = request.workspace     # কখনো ফর্ম থেকে নয়
obj.save()
```

fields কেন দরকার

`"__all__"` দিলে ব্যবহারকারী নিজের `workspace`, `status` — এমনকি `id` পর্যন্ত পাঠাতে পারে।

workspace ফর্মে নেই

তালিকায় নেই মানে POST-এ পাঠানোও যায় না — mass assignment বন্ধ।

widgets শুধু চেহারা

`type="date"` দিলে ব্রাউজারে তারিখ-নির্বাচক আসে — নিয়ম নয়, সুবিধা।

শিক্ষকের নোট: আক্রমণটা দেখান: ব্রাউজারের ডেভ-টুলে একটি লুকানো ইনপুট যোগ করে `workspace=2` পাঠান — `fields`-এ না থাকলে কিছুই হয় না। এটাই সবচেয়ে বাস্তব নিরাপত্তা-পাঠ।

ধাপ ৩

একটি field-level নিয়ম, তিনটি সূক্ষ্মতা

`clean_admission_no()` — ইউনিক, কিন্তু সম্পাদনার সময় নিজেকে বাদ দিয়ে

```
def clean_admission_no(self):
    value = self.cleaned_data["admission_no"].strip()
    qs = StudentProfile.objects.filter(
        workspace=self.workspace,
        admission_no__iexact=value)
    if self.instance.pk:          # সম্পাদনা করছি
        qs = qs.exclude(pk=self.instance.pk)
    if qs.exists():
        raise forms.ValidationError(
            "Another student in this workspace "
            "already uses this admission number.")
    return value
```

সূক্ষ্মতা	না করলে
`workspace=` ফিল্টার	অন্য প্রতিষ্ঠানের নম্বরও "দখল" দেখাবে
`__iexact`	`A-12` আর `a-12` দুইবার ঢুকবে
`.strip()`	শেষে স্পেস থাকলে ভিন্ন মান
`exclude(pk=…)`	নিজের রেকর্ড সম্পাদনা করতে গেলে "আগেই আছে" এরর
`return value`	পরের ধাপে `None` পাবে — ডেটা হারাবে

শেষ সারিটি সবচেয়ে বেশি ভুল হয়: `clean_<field>` কখনো `None` ফেরাতেই হবে না — **পরিষ্কার করা মান** ফেরাতে হয়।

শিক্ষকের নোট: "সম্পাদনা করুন" আর "নতুন যোগ করুন" — একই ফর্ম। ছাত্ররা exclude(pk) ভুলে যায়, তারপর বলে "আমার ফর্মটা কাজ করে না।" এখানেই সেই বাগের জন্ম।

ধাপ ৪

দুই mixin, একই কাজ — একটি সততার পাঠ

কোডবেসে সবকিছু নিখুঁত নয়; চেনা ও ঠিক করার ক্ষমতাই আসল দক্ষতা

```python
# apps/academics/forms.py
class BootstrapFormMixin:
    def __init__(self, *args, **kwargs):
        super().__init__(*args, **kwargs)
        for name, field in self.fields.items():
            w = field.widget
            if isinstance(w, forms.CheckboxInput):
                w.attrs.setdefault("class", "form-check-input")
            elif isinstance(w, forms.Select):
                w.attrs.setdefault("class", "form-select")
            else:
                w.attrs.setdefault("class", "form-control")

# apps/billing/forms.py — একই কাজ, ভিন্ন নাম
class ImsFormMixin:
    def __init__(self, *args, **kwargs):
        super().__init__(*args, **kwargs)
        for field in self.fields.values():
            widget = field.widget
            css = widget.attrs.get("class", "")
            if widget.__class__.__name__ == "CheckboxInput":
                widget.attrs["class"] = (css + " form-check-input").strip()
            ... আরও চারটি শাখা
```

সমস্যাটা কী

দুটি ফাইল, দুটি নাম, প্রায় একই কাজ — একটি বদলালে অন্যটি পুরোনো হয়ে থাকবে।

লক্ষণ

কিছু পর্দায় তারিখ-নির্বাচক বা রঙ-নির্বাচক অন্যভাবে দেখায়।

সমাধান

একটি mixin `apps/core/`-এ সরান, অন্যটি ইমপোর্ট করুন — দুই লাইনেই কাজ।

শিক্ষা

প্রতিলিপি কোড নিজে থেকে ভাঙে না — **একদিন একটি কপি আপডেট হয়**, তখন ভাঙে।

শিক্ষকের নোট: এটা স্বীকার করুন: "এই ডেকে আমি IMS-এর ভুলও দেখাই।" ছাত্রদের বলুন নিজের কোডে grep করে দেখতে — `Bootstrap`, `Mixin`, `mixin`।

ধাপ ৫

ড্রপডাউনের ভেতরটা — queryset কে ঠিক করে

`__init__`-এ queryset বদলানো, আর কখনোই `objects.all()` নয়

```
class EnrolForm(forms.Form):
    students = forms.ModelMultipleChoiceField(
        queryset=StudentProfile.objects.none(),   # নিরাপদ শুরু
        widget=forms.CheckboxSelectMultiple(
            attrs={"class": "form-check-input"}),
        help_text="Only active students not already in "
                  "this class are listed.")

    def __init__(self, *args, workspace=None,
                 class_group=None, **kwargs):
        super().__init__(*args, **kwargs)
        qs = StudentProfile.objects.filter(
            workspace=workspace,
            status=StudentProfile.Status.ACTIVE)
        if class_group is not None:
            already = Enrollment.objects.filter(
                class_group=class_group, is_active=True,
            ).values_list("student_id", flat=True)
            qs = qs.exclude(pk__in=already)
        self.fields["students"].queryset = qs.order_by("full_name")
```

`none()` দিয়ে শুরু

কেউ `workspace` দিতে ভুলে গেলে খালি তালিকা — ফাঁস নয়।

আগেই ভর্তি কারা

`exclude(pk__in=already)` — তালিকায় দেখাবে না, আর জমা দিলে ভ্যালিডেশনেও আটকাবে।

দুই স্তরে একই নিয়ম

তালিকা সাজানো = UX · queryset = নিরাপত্তা। দুটোই লাগে।

শিক্ষকের নোট: প্রশ্ন করুন: "ড্রপডাউনে না দেখালে কি কেউ পাঠাতে পারবে?" উত্তর: হ্যাঁ, হাতে POST বানিয়ে — তাই queryset-ই আসল প্রহরী, তালিকাটি নয়।

ধাপ ৬

দুই ফিল্ডের সম্পর্ক — `clean()` আর `add_error()`

`BrandingForm`: ডোমেইন ভেরিফাই করা যাবে, কিন্তু খালি ডোমেইনে নয়

```python
def clean(self):
    data = super().clean()
    if data.get("domain_verified") and not data.get("custom_domain"):
        self.add_error("domain_verified",
                       "Verify a domain only after entering one.")
    return data

# কল্পিত খারাপ সংস্করণ
def clean(self):
    if self.cleaned_data["domain_verified"]:
        raise forms.ValidationError("...")
    return self.cleaned_data
```

add_error(field, ...)

এররটা ঠিক ওই ঘরের নিচে বসে — ব্যবহারকারী এক নজরে দেখেন।

raise ValidationError

উপরের সারাংশে যায় — কোন ঘর ঠিক করবেন বোঝা কঠিন।

super().clean() আগে

নাহলে `cleaned_data`-তে ফিল্ড না-ও থাকতে পারে — KeyError।

একই প্যাটার্ন অন্য জায়গায়

`InstitutionForm.clean_admin_email` — ইমেইলের অ্যাকাউন্ট আগেই আছে কি না।

শিক্ষকের নোট: বলুন: "এরর কোথায় দেখাবে, সেটাও ডিজাইনের অংশ। উপরে সারাংশ দেখালে ব্যবহারকারী ২০টি ঘরের মধ্যে খুঁজতে থাকেন।"

ধাপ ৭

এরর থেকে পর্দা — এক partial, সব ফর্ম

`partials/field.html` — লেবেল, সহায়তা ও এরর একই জায়গায়

```
{% include "partials/field.html"
   with field=form.username %}

{# field.html যা করে #}
<label>{{ field.label }}
  {% if field.field.required %}*{% endif %}</label>
{{ field }}
{% if field.help_text %}
  <div class="ims-hint">{{ field.help_text }}</div>
{% endif %}
{% for error in field.errors %}
  <div class="ims-error">{{ error }}</div>
{% endfor %}
```

কোথায় কী দেখায়	উৎস
ফিল্ডের নিচের এরর	`field.errors`
ফর্মের সার্বিক এরর	`form.non_field_errors`
ধূসর সহায়ক লেখা	`help_text`
সবুজ সেভ বার্তা	`messages.success`
লাল ফ্ল্যাশ	`messages.error`

লগইনে `non_field_errors` উপরে দেখানো হয় — কারণ "ইমেইল" না "পাসওয়ার্ড" ভুল, তা বলা যায় না।

শিক্ষকের নোট: শেষ কথাটা গুরুত্বপূর্ণ: "কোনটা ভুল তা বলা মানে আক্রমণকারীকে বলা 'ইমেইলটা ঠিক আছে'।" তাই লগইনে সবসময় সাধারণ বার্তা।

ধাপ ৮

যেখানে প্রতি সারি একটি ফিল্ড — তবু ফর্ম

`AttendanceBulkForm.statuses_for()` — ৪০ জন ছাত্র, ৪০টি রেডিও

```
class AttendanceBulkForm(forms.Form):
    class_group = forms.ModelChoiceField(...)
    subject = forms.ModelChoiceField(...)
    date = forms.DateField(widget=forms.DateInput(
        attrs={"type": "date"}))

    def statuses_for(self):
        """Read the per-student radio values."""
        data = {}
        for key, value in self.data.items():
            if key.startswith("status_"):
                data[int(key.split("_")[1])] = value
        return data

# ভিউতে যাচাই আবার হয়
if status not in dict(AttendanceRecord.Status.choices):
    continue
```

কেন ডাইনামিক

ছাত্রসংখ্যা প্রতি ক্লাসে বদলায় — ফর্ম ক্লাসে ৪০টি ফিল্ড লিখে রাখা অসম্ভব।

নামকরণই চুক্তি

`status_<student_id>` — টেমপ্লেট ও ফর্ম একই নাম মানে।

ভুলের ঝুঁকি

কোনো ছাত্রের রেডিও বাদ পড়লে সে "অনুপস্থিত" নয় — **রেকর্ডই হয় না**। তাই ভিউ সেভের আগে যাচাই করে।

বিকল্প

ফর্মset বা প্রতি সারির জন্য আলাদা ফর্ম — জটিল, কিন্তু সব ভ্যালিডেশন স্বয়ংক্রিয়।

শিক্ষকের নোট: এখানে সীমাবদ্ধতাটা পরিষ্কার বলুন: "ডাইনামিক ফিল্ড মানে ভ্যালিডেশনের দায়িত্ব আংশিক আপনার — ফর্ম আর সব জানে না।"

ধাপ ৯

পাসওয়ার্ড — একটি ফর্ম নয়, চারটি নিয়ম

`AUTH_PASSWORD_VALIDATORS` — Django নিজেই চারটি যাচাই করে

ভ্যালিডেটর	নিয়ম	কী আটকায়
UserAttributeSimilarity	নাম/ইমেইলের মতো নয়	`admin@ims.local` → `admin123`
MinimumLengthValidator	**কমপক্ষে ১০ অক্ষর**	ছোট পাসওয়ার্ড
CommonPasswordValidator	তালিকাভুক্ত সাধারণ নয়	`password` · `qwerty123`
NumericPasswordValidator	শুধু সংখ্যা নয়	`01711111111`

ফর্মে: `min_length=10` · `PasswordInput` · `help_text`-এ নিয়ম লেখা।

ডেমো পাসওয়ার্ড `DemoPass!2026` — ১১ অক্ষর, তাই চারটি নিয়মই পাশ করে।

ফর্মের `min_length` আর ভ্যালিডেটর — দুটোই দরকার: প্রথমটি ভদ্র বার্তা দেয়, দ্বিতীয়টি **অন্য সব পথেও** কাজ করে (অ্যাডমিন, শেল, রিসেট)।

শিক্ষকের নোট: `python manage.py check --deploy` চালিয়ে দেখান — পাসওয়ার্ড ভ্যালিডেটর বাদ দিলে সেটাও সতর্ক করে।

সততার ফাঁক

যেটা IMS-এ এখনো নেই: আপলোড যাচাই

লোগো ফিল্ড শুধু `help_text`-এ বলে "PNG or SVG" — কোড কিছু যাচাই করে না

```
# যা আছে (মডেলের নির্দেশনা মাত্র)
logo = models.ImageField(
    upload_to="branding/logos/", blank=True, null=True,
    help_text="PNG or SVG, ideally 512x512 or wider.")

# যা যোগ করা উচিত
def clean_logo(self):
    logo = self.cleaned_data.get("logo")
    if logo and hasattr(logo, "size"):
        if logo.size > 2 * 1024 * 1024:
            raise forms.ValidationError(
                "Logo must be under 2 MB.")
        ext = logo.name.rsplit(".", 1)[-1].lower()
        if ext not in ("png", "jpg", "jpeg", "svg", "webp"):
            raise forms.ValidationError(
                "Use PNG, JPG, SVG or WebP.")
    return logo
```

ঝুঁকি ১: আকার

৫০ MB লোগো আপলোড হলে ডিস্ক ও ব্যাকআপ ফুলে যায়।

ঝুঁকি ২: ধরন

HTML/SVG স্ক্রিপ্টসহ আপলোড হলে XSS-এর দরজা খোলে — SVG-তে JS থাকতে পারে।

ঝুঁকি ৩: নাম

ব্যবহারকারীর দেওয়া ফাইলনাম সরাসরি ব্যবহার করবেন না — Django নিজেই নাম বদলায়।

পাঠ

"যা লিখিনি, তা হয়নি।" `help_text` নিয়ম নয়, অনুরোধ।

শিক্ষকের নোট: এই স্লাইডটি ইচ্ছাকৃতভাবে সৎ — বলুন: "বাস্তব প্রজেক্টে এমন ফাঁক থাকে। চেনা, লিখে রাখা, পরে ঠিক করা — এটাই পেশাদারি।"

সতর্কতা

ফর্মের পাঁচটি সাধারণ ভুল

প্রতিটির প্রতিকার এক লাইনে

ভুল	পরিণতি ও প্রতিকার
১. `fields = "__all__"`	ব্যবহারকারী নিজের স্কোপ/স্ট্যাটাস পাঠাতে পারে → নির্দিষ্ট তালিকা
২. `queryset=Model.objects.all()`	অন্য প্রতিষ্ঠানের নাম ড্রপডাউনে → `none()` + `__init__`-এ স্কোপ
৩. `clean_<field>` মান ফেরাত না দেওয়া	পরের ধাপে `None` → `return value`
৪. সব এরর `raise` দিয়ে	ভুল ঘর বোঝা যায় না → `add_error("field", …)`
৫. ফর্মের ভ্যালিডেশনকে নিরাপত্তা ভাবা	অন্য পথ (শেল, ইমপোর্ট) খোলা থাকে → মডেল constraint-ও রাখুন

সবগুলো এক প্রশ্নে ধরা পড়ে: **"এই মানটা কি অন্য কোনো পথে সেভ হতে পারে?"** — হ্যাঁ বললে ফর্ম একা যথেষ্ট নয়।

শিক্ষকের নোট: পঞ্চম ভুলটি ক্লাস ১৬-এর সাথে জোড়া লাগান: "ফর্ম ভদ্র প্রহরী, constraint শেষ দেয়াল।"

অনুশীলন

২০ মিনিট — নিজের একটি ফর্ম শক্ত করুন

একই ফর্মে পাঁচটি স্তর

১. হোয়াইটলিস্ট (৩ মিনিট)

একটি ModelForm-এ `fields` তালিকা লিখুন — `__all__` থাকলে বাদ দিন।

২. স্কোপড queryset (৫ মিনিট)

প্রতিটি ModelChoiceField-এ `none()` + `__init__`-এ workspace ফিল্টার।

৩. একটি field-level নিয়ম (৫ মিনিট)

`clean_<field>` — unique চেক, `iexact`, `exclude(pk)`, `return value`।

৪. একটি cross-field নিয়ম (৪ মিনিট)

`clean()`-এ দুই ফিল্ডের সম্পর্ক, `add_error()` দিয়ে নির্দিষ্ট ঘরে।

৫. যাচাই (৩ মিনিট)

ব্রাউজারের ভ্যালিডেশন বন্ধ করে ফাঁকা ফর্ম পাঠান — এররগুলো ঠিক ঘরে আসছে?

শিক্ষকের নোট: `novalidate`-টা লাইভে দেখান — অনেকের কাছে এটাই সবচেয়ে চোখ খোলা মুহূর্ত: "ব্রাউজার যেটা আটকাচ্ছিল, সেটা আটকাচ্ছিল না আসলে।"

আলোচনা

ক্লাসে আলোচনার চার প্রশ্ন

উত্তর এক বাক্যে

- একই ইউনিক-চেক কেন সম্পাদনার সময় নিজেকে বাদ দিতে হয়?
- ড্রপডাউনে না দেখিয়ে দিলেও কেন কেউ মান পাঠাতে পারে?
- এরর সব উপরে দেখালে ব্যবহারকারীর কী অসুবিধা?
- ফর্মের নিয়ম থাকলেও মডেলে নিয়ম রাখতে হয় কেন?

শিক্ষকের নোট: দ্বিতীয় প্রশ্নে হাতে POST দেখানোর সুযোগ নিন (curl বা ডেভ-টুল) — বোঝা যায় UI কখনো নিরাপত্তা নয়।

সারমর্ম

ছয়টি কথা সঙ্গে নিন

প্রতিটির পাশে IMS-এর প্রমাণ

নীতি	প্রমাণ
fields হলো হোয়াইটলিস্ট	`StudentForm`-এ workspace নেই
queryset `none()` দিয়ে শুরু	`EnrolForm` · `AttendanceBulkForm`
clean চেক স্কোপ + iexact + exclude(pk)	`clean_admission_no`
এরর নির্দিষ্ট ঘরে	`add_error("domain_verified", …)`
এক partial, সব ফর্মের চেহারা	`partials/field.html` · ৩৫ ফর্ম
পাসওয়ার্ড নিয়ম ফর্মে + ভ্যালিডেটরে	৪টি validator · `min_length=10`

শিক্ষকের নোট: প্রথম ও তৃতীয় সারি সবচেয়ে বেশি কাজে লাগে — বলুন: "এই দুটো অভ্যাস থাকলে আপনার ৮০% ফর্ম-বাগ উঠে যাবে।"

১৯

পরের ক্লাস — "ডেটা থেকে পর্দা"

টেমপ্লেট: inheritance · context · loop ও empty · custom filter · partial · template-এ লজিকের সীমা

পর্ব ৪ · কোডিং · ক্লাস ১৯ / ৩৭

ডেটা থেকে পর্দা

সিলেবাস: প্র্যাকটিক্যাল ৬ · টপিক ৬.১ (টেমপ্লেট)

৭৫টি টেমপ্লেট · ৪টি শেল · ৬টি partial · ১৫টি ডিজাইন টোকেন

inheritance · context processor · empty state

শুরু করুন প্রশ্ন দিয়ে

৭৫টি ফাইল — তবু ক'টি শেল?

হাত তুলুন: নতুন একটি পাতা বানাতে কতটা HTML লিখতে হয়?

উত্তর: প্রায় শূন্য — শুধু `{% block content %}`-এর ভেতরটা।

sidebar, topbar, মেসেজ, বিলিং ব্যানার — সব একবার লেখা, **৪৪টি পাতা** সেটাই ব্যবহার করে। আজকের ক্লাসে দেখব এই একবার-লেখার কারিগরি, আর তার তিনটি ফাঁদ।

শিক্ষকের নোট: "একটি পাতা বানাতে ২০০ লাইন HTML লেখা মানে ৪৪ বার একই ভুল করার সুযোগ।" এটাই template inheritance-এর পুরো যুক্তি।

ধাপ ১

চারটি খোলস, ৭৫টি পাতা

গোনা হয়েছে `{% extends %}` দিয়ে — ৬৫টি পাতার ঠিকানা

base.html

head · body · scripts

৩টি ব্লক, ০টি পাতা

লগইন ৬ + এরর ৩ পাতা

সরাসরি এর ওপর দাঁড়ায়

app_base.html

sidebar + topbar + content

৫টি include

৪৪টি পাতা

কাজের সব পর্দা এখানে

platform/base.html

৬টি ট্যাব, ভিন্ন সাইডবার

টেন্যান্সির বাইরে

৮টি পাতা

প্ল্যাটফর্ম কনসোল

public/base_public

মার্কেটিং শেল

লগইন ছাড়াই দেখা যায়

২টি পাতা

pricing · signup

templates/partials/ — ৬টি টুকরো: sidebar · topbar · messages · billing_banner · cookie_guard · field

চারটি শেল থাকা মানে চারটি আলাদা জীবনের ধরন: লগইন করা ব্যবহারকারী · প্ল্যাটফর্ম অ্যাডমিন · বাইরের অতিথি · আর শুধুই কাঠামো।

শিক্ষকের নোট: ব্ল্যাকবোর্ডে চারটি নাম লিখুন, তারপর জিজ্ঞেস করুন বিলিং পর্দা কোন শেলে — উত্তর app_base। "নতুন শেল কখন? যখন ব্যবহারকারীর জীবনটাই আলাদা।"

ধাপ ২

৪৪টি পাতার একটাই বাবা — `app_base.html`

১২ লাইন HTML, তার ভেতরেই পুরো অ্যাপের কাঠামো

```
{% extends "base.html" %}
{% block body %}
<div class="ims-shell">
  {% include "partials/sidebar.html" %}
  <div class="ims-main">
    {% include "partials/topbar.html" %}
    <div class="ims-content">
      {% include "partials/messages.html" %}
      {% include "partials/billing_banner.html" %}
      {% block content %}{% endblock %}
    </div>
  </div>
</div>
{% endblock %}
```

পর্দার ক্রমটাই এখানে ঠিক হয়: বার্তা → ব্যানার → আসল কাজ।

ব্লক	কত পাতা ভরে
`title`	**৬৫** — ট্যাবের নাম
`content`	**৫৪** — আসল কাজ
`body`	**১৫** — শেল বদলায়
`scripts`	**২** — চার্ট/এডিটর
`heading`	**১** — topbar-এর শিরোনাম

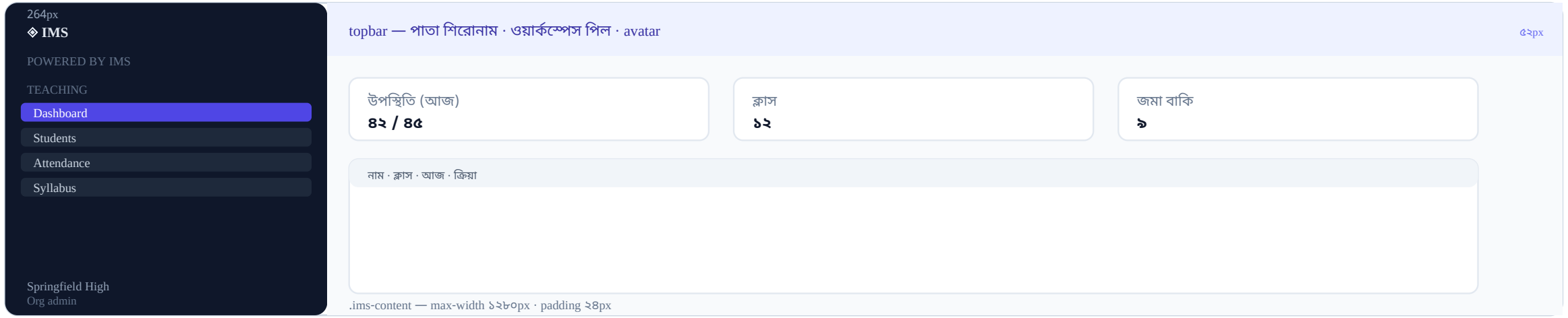
`{% block %}` গুলো সবসময় **একই নামে** থাকতে হবে — নাম বদলালে ৫৪টি পাতায় ওই জায়গাটা ফাঁকা হয়ে যাবে, কোনো এরর ছাড়াই।

শিক্ষকের নোট: শেষ সতর্কতাটা লাইভে দেখান: ব্লকের নাম এক অক্ষর বদলে রেন্ডার করুন — কোনো এরর নেই, শুধু লেখা গায়েব। "নীরব ব্যর্থতা সবচেয়ে খারাপ ব্যর্থতা।"

ধাপ ৩

শেলটা কেমন — মাপসহ

`.ims-shell` = `grid-template-columns: 264px 1fr`



`@media (max-width:820px)` → এক কলাম, সাইডবার লুকিয়ে যায়।

প্রিন্ট/মোবাইলে সাইডবার ছাড়া **ন্যাভিগেশনই থাকে না** — IMS-এ এটি এখনো খোলা প্রশ্ন।

শিক্ষকের নোট: ব্রাউজারে ডিভ-টুল খুলে `264px` বদলিয়ে দেখান — সব ৪৪ পাতায় একসাথে বদলায়। এটাই "একবার ঠিক করুন" নীতির চোখে-দেখা প্রমাণ।

ধাপ ৪

১৫টি টোকেন, পুরো প্রোডাক্টের চেহারা

`static/css/ims.css`-এর `:root` — ৬০০ লাইনের কোড একই ভাষায় কথা বলে

```
:root{
  --ims-primary:#4f46e5; --ims-primary-dark:#4338ca;
  --ims-primary-soft:#eef2ff;
  --ims-ink:#0f172a; --ims-ink-2:#334155;
  --ims-muted:#64748b;
  --ims-line:#e2e8f0; --ims-bg:#f6f7fb;
  --ims-surface:#ffffff;
  --ims-success:#059669; --ims-warning:#d97706;
  --ims-danger:#dc2626;
  --ims-radius:14px; --ims-radius-sm:10px;
  --ims-shadow:0 1px 2px rgba(15,23,42,.06),
               0 8px 24px -12px rgba(15,23,42,.18);
}
```

কেন টোকেন

রঙ এক জায়গায় — ৪৪ পাতায় `#4f46e5` খুঁজতে হয় না।

শৃঙ্খলা

সব সাদা পৃষ্ঠ `--ims-surface`, সব বর্ডার `--ims-line` — নতুন পর্দাও পুরোনোর মতো দেখায়।

হোয়াইট-লেবেলের দরজা

টোকেন থাকায় একটি ভেরিয়েবল বদলেই প্রতিষ্ঠানের রঙ বসানো যায় (পরের দিকে)।

শিক্ষকের নোট: বলুন: "টোকেন হলো ডিজাইনের ভোকাবুলারি। টোকেন না থাকলে প্রতিটি পর্দা নিজের ভাষায় কথা বলে, আর প্রোডাক্ট দেখতে সেলাই-করা কম্বল লাগে।"

ধাপ ৫

একই ড্যাশবোর্ড, তিনটি পর্দা

ভিউ-তে `if role`, টেমপ্লেটে আলাদা ভাগ — এক URL, ভিন্ন অভিজ্ঞতা

ভূমিকা	টেমপ্লেট	pর্দায় কী থাকে
STUDENT	`dashboard_student.html`	উপস্থিতি % · প্রকাশিত পরীক্ষা · খোলা হোমওয়ার্ক (৫) · জমার সংখ্যা
TEACHER	`dashboard_staff.html`	আজকের উপস্থিতি · ক্লাস · অপ্রকাশিত পরীক্ষা · জমা বাকি · সাম্প্রতিক পরীক্ষা (৬)
COORDINATOR / ORG_ADMIN	একই `dashboard_staff.html`	উপরের সব + **সদস্য সংখ্যা · অডিট লগ (৮) · উপস্থিতি নীতি**
PLATFORM_ADMIN	`core/dashboard_platform.html`	প্রতিষ্ঠান · MRR · ট্রায়াল · সাসপেন্ড

`if role in MANAGERS` — একই টেমপ্লেট, বাড়তি কী-তে বাড়তি ব্লক।

`core/`-এ **৪টি ড্যাশবোর্ড টেমপ্লেট** আছে যেগুলো **কোথাও রেন্ডার হয় না** — মৃত ফাইল, শুধু বিভ্রান্তি।

শিক্ষকের নোট: "মৃত টেমপ্লেট খুঁজে বের করা শেখা একটা বাস্তব দক্ষতা" — grep করে দেখান: ভিউয়ের `render()`-এ ওই ফাইলগুলোর নাম নেই।

ধাপ ৬

২৩টি মেনু আইটেম, একটাই সংজ্ঞা

`apps/core/navigation.py` — এক ফাইল, তিন ভোক্তা

```
NAV = [
  {"title": None, "items": [
     _item("Dashboard", "bi-speedometer2",
           "reports:dashboard", STAFF+(STUDENT,))]},
  {"title": "Teaching", "items": [
     _item("Students", "bi-people",
           "students:list", STAFF_ROLES),
     _item("Attendance", "bi-calendar-check",
           "attendance:take", STAFF_ROLES), ...]}]

# রেন্ডারের সময় ফিল্টার
for item in group["items"]:
    if role not in item["roles"]:
        continue
    url = reverse(item["url_name"], kwargs={"slug": slug})

# sidebar-এ সক্রিয় লিংক
class="ims-nav {% if request.path == item.url
  or item.url in request.get_full_path %}is-active{% endif %}"
```

তিন ভোক্তা

সাইডবার দেখায় · ড্যাশবোর্ড হাইলাইট করে · টেস্ট যাচাই করে।

ব্যর্থতার ধরন

নতুন পাতা যোগ করে এই ফাইল ভুলে গেলে — **পাতাটা মেনুতে থাকে না**, কিন্তু URL কাজ করে।

নিরাপত্তার সীমা

মেনু লুকানো **অনুমতি নয়** — সার্ভার প্রতিটি অনুরোধে আলাদা যাচাই করে (ক্লাস ১৭)।

একটি সুক্ষ্ম বাগ

`item.url in request.get_full_path` — সাবস্ট্রিং মিল, তাই কখনো ভুল আইটেমও হাইলাইট হতে পারে।

শিক্ষকের নোট: শেষ কার্ডটি নিয়ে ছাত্রদের ভাবতে দিন: "two" আর "twelve"-এর মতো। সমাধান সাধারণত `startswith` বা সুস্পষ্ট prefix।

ধাপ ৭

খালি তালিকাও একটা পর্দা

১৬টি `{% empty %}` · ৪১টি ফাইলে "empty" — খালি অবস্থা আলাদা করে ভাবা হয়েছে

```
{% for scale in scales %}
  <div class="ims-card">...</div>
{% empty %}
  <div class="ims-card"><div class="ims-empty">
    <i class="bi bi-list-check"></i>
    No grade scale yet.
  </div></div>
{% endfor %}

<tr><td colspan="6" class="ims-hint">
  No invoice matches this filter.
</td></tr>
```

দুটি ভিন্ন বার্তা: **"এখনো কিছু নেই"** আর **"এই ফিল্টারে কিছু নেই"** — ব্যবহারকারীর পরের কাজ আলাদা।

নীতি	কেন
এক আইকন + এক বাক্য	লম্বা ব্যাখ্যা পড়া হয় না
পরের ধাপের লিংক	"নতুন স্কেল" — খালি পর্দা থেকে বেরোনোর পথ
এরর থেকে আলাদা	খালি মানে ভুল নয়, শুধু এখনো কিছু নেই
টেবিলে `colspan`	নাহলে খালি সারি অদ্ভুত দেখায়

সবচেয়ে বড় ভুল: `{% empty %}` না লেখা — ব্যবহারকারী ভাবেন সাইট ভেঙে গেছে।

শিক্ষকের নোট: লাইভে ডেমো: নতুন প্রতিষ্ঠান খুলে খালি ড্যাশবোর্ড দেখান। "প্রথম দিনের ব্যবহারকারী এটাই দেখেন — এটাই আপনার প্রথম ছাপ।"

ধাপ ৮

টেমপ্লেটে লজিক — কতটুকু?

Django ইচ্ছাকৃতভাবে দুর্বল টেমপ্লেট ভাষা দেয়; IMS-এ **০টি** নিজস্ব templatetag

```
{# messages.html — শুধু চেহারা ঠিক করা #}
{% if message.tags == 'success' %}bi-check-circle
{% elif message.tags == 'danger' %}bi-exclamation-octagon
{% elif message.tags == 'warning' %}bi-exclamation-triangle
{% else %}bi-info-circle{% endif %}

{# একই ফাইলে ফিল্টারের কাজ #}
{{ message.tags|default:'info' }}
{{ subscription.days_left }} day{{ subscription.days_left|pluralize }}
{{ request.user.full_name|first|upper }}
{{ brand.color|default:"#4f46e5" }}
```

ফিল্টার	কাজ
`default`	খালি হলে ফলব্যাক রঙ/মান
`pluralize`	"1 day" / "5 days"
`first|upper`	avatar-এর অক্ষর
`date` · `floatformat`	তারিখ ও টাকার অঙ্ক

হিসাব, অনুমতি, কুয়েরি — ভিউতে। টেমপ্লেটে শুধু **দেখানোর সিদ্ধান্ত**।

`templatetags/` ফোল্ডার কোথাও নেই — কারণ যোগ করলে লজিক দুই জায়গায় ছড়িয়ে পড়ে।

শিক্ষকের নোট: প্রশ্ন করুন: "শতাংশের হিসাব টেমপ্লেটে করলে কী ক্ষতি?" উত্তর: টেস্ট লেখা যায় না, আর দুই পর্দায় দুই রকম হিসাব হয়ে যায়।

ধাপ ৯

তিনটি টুকরো, ৩৬টি জায়গায়

`{% include %}` — ৩৬ বার, সবচেয়ে বেশি `field.html`

```
{# partials/field.html #}
<div class="mb-3">
  <label for="{{ field.id_for_label }}">
    {{ field.label }}{% if field.field.required %}
    <span class="text-danger">*</span>{% endif %}
  </label>
  {{ field }}
  {% if field.help_text %}
    <div class="ims-hint">{{ field.help_text }}</div>
  {% endif %}
  {% for error in field.errors %}
    <div class="ims-error">{{ error }}</div>
  {% endfor %}
</div>
```

এক লাইনে ব্যবহার

`{% include "partials/field.html" with field=form.username %}`

লুপেও চলে

গ্রেড স্কেলে: `{% for field in form %}` → প্রতিটি ঘর একই চেহারা।

বাকি দুটি

`messages.html` (ফ্ল্যাশ বার্তা) · `sidebar.html` + `topbar.html` (শেল)।

include বনাম extends

include = ভেতরে বসানো টুকরো (context পায়); extends = উত্তরাধিকার (ব্লক ভরে)। গুলিয়ে ফেলা সাধারণ ভুল।

শিক্ষকের নোট: শেষ কার্ডটা বোর্ডে দুটি ছবি এঁকে বোঝান: একটি বাক্স আরেক বাক্সের ভেতরে (include), বনাম একটি কাঠামো যার ফাঁকা ঘর ভরা হচ্ছে (extends)।

ধাপ ১০

প্রতিষ্ঠানের নিজের রঙ — ৪ লাইনে

`base.html`-এর ইনলাইন `:root` ওভাররাইড — সব ৭৫ পাতায় একসাথে

```
<style>
  :root {
    --ims-brand: {{ brand.color|default:"#4f46e5" }};
  }
  .btn-ims, .ims-nav.is-active,
  .ims-auth__submit { background: var(--ims-brand); }
  .ims-sidebar {
    background: linear-gradient(180deg,
      color-mix(in srgb, var(--ims-brand) 22%, #0b1220),
      #0b1220); }
  a { color: var(--ims-brand); }
</style>

# context processor — brand একটি dict
brand = {"name": row.brand_name, "logo": row.logo.url,
         "color": row.primary_color or "#4f46e5",
         "initial": row.initial, "tagline": row.tagline,
         "login_message": row.login_message,
         "support_email": row.support_email}
```

কী	কোথায় লাগে
`brand.name`	১১ জায়গায় — টাইটেল, সাইডবার, লগইন
`brand.color`	৩ — বাটন, সাইডবার, লিংক
`brand.logo`	৩ — সাইডবার, লগইন, ইনভয়েস
`brand.support_email`	৩ — সহায়তা লিংক
`product.name / short`	৮ — "POWERED BY" লাইন

dict কেন, মডেল কেন নয়? ব্র্যান্ডিং সারি না থাকলে (নতুন ওয়ার্কস্পেস) টেমপ্লেটে **কোনো এররই উঠবে না**।

ঝুঁকি: প্রতিষ্ঠান হালকা রঙ বাছলে সাদা লেখা পড়া যায় না — কনট্রাস্ট ভ্যালিডেশন নেই।

শিক্ষকের নোট: ডেমোতে রঙ বদলে সেভ করুন — পুরো অ্যাপ বদলে যায়। তারপর "লাল রঙ দিলে ডেঞ্জার বাটন আর আলাদা করা যায় না" — এই সীমাটা বলুন।

ধাপ ১১

ব্যানার — সাবস্ক্রিপশনের কণ্ঠস্বর

এক partial, চারটি অবস্থা; কাজের লিংক শুধু `ORG_ADMIN`-এর জন্য

শর্ত	রঙ	ব্যবহারকারী যা দেখেন
`TRIAL` + writable	নীল	"Free trial — N দিন বাকি (plan, ends …)" → Choose a plan
writable নয়, SUSPENDED নয়	হলুদ	"Read-only mode … renew to start editing again" → Renew now
`SUSPENDED`	হলুদ	"Account suspended — data kept and readable" (লিংক নেই, ইচ্ছাকৃত)
সিট ≥ ৯০%	হলুদ	"৪২/৪৫ seats used" → Upgrade

সব ব্যানারের ভেতরে `{% if nav_role == "ORG_ADMIN" %}` — সাধারণ শিক্ষক লিংক দেখেন না, কারণ তিনি কিছু করতে পারেন না।

ব্যানার কী-তে নয়, শেলেই — তাই **৪৪টি পাতার প্রতিটিতে** একই সত্য দেখায়, কেউ ভুলে যায় না।

ব্যানার শুধু জানায়, আটকায় না — লেখা বন্ধ করা হয় মডেল/ভিউ স্তরে (`is_writable`)। UI বার্তা আর নিয়ম আলাদা জিনিস।

শিক্ষকের নোট: `bash run.sh` চালিয়ে ডেটাবেসে ট্রায়াল শেষ করুন — ব্যানার বদলাতে দেখুন। "ব্যবহারকারীকে হঠাৎ অবাক করবেন না" — এটাই এই partial-এর একমাত্র কাজ।

অনুশীলন ও আলোচনা

২০ মিনিট — আর চারটি প্রশ্ন

নিজের প্রজেক্টে তিনটি শেল, তারপর আলোচনা

১. তিনটি শেল (৮ মিনিট)

লগইন · কাজের পর্দা · পাবলিক পাতা — তিনটি base লিখুন, প্রতিটিতে `{% block content %}`।

২. দুটি partial (৭ মিনিট)

`_nav.html` ও `_messages.html` বানিয়ে দুই পাতায় include করুন।

৩. খালি অবস্থা (৫ মিনিট)

একটি লুপে `{% empty %}` যোগ করুন — আইকন + এক বাক্য + পরের ধাপের লিংক।

- একটি ব্লকের নাম বদলালে কেন কোনো এরর আসে না?
- মেনুতে লুকানো থাকলেই কি অনুমতি দেওয়া হয়েছে?
- খালি তালিকা আর এরর — ব্যবহারকারীর কাছে কেন আলাদা?
- ব্র্যান্ড রঙ ব্যবহারকারীর হাতে দিলে কী ঝুঁকি?

শিক্ষকের নোট: তৃতীয় প্রশ্নের সেরা উত্তর: "খালি মানে 'এখনো নয়', এরর মানে 'হয়নি'।" প্রথম প্রশ্নের উত্তর— Django ব্লক না পেলে চুপচাপ খালি রেন্ডার করে।

সারমর্ম

ছয়টি কথা সঙ্গে নিন

প্রতিটির পাশে IMS-এর প্রমাণ

নীতি	প্রমাণ
চারটি শেল, ৬৫টি extends	`app_base` ৪৪ · `base` ১১ · platform ৮ · public ২
এক শেলেই সব সাধারণ জিনিস	`app_base.html` — ১২ লাইন, ৫টি include
চেহারা টোকেনে, যুক্তি ভিউতে	১৫টি CSS ভেরিয়েবল · ০টি templatetag
ভূমিকা বুঝে পর্দা	৩টি জীবিত ড্যাশবোর্ড · `if role in MANAGERS`
খালিও ডিজাইন করা	১৬টি `{% empty %}` · `.ims-empty`
মেনু দেখানো ≠ অনুমতি	`navigation.py` + `@workspace_view`

শিক্ষকের নোট: ষষ্ঠ সারিটা আবার জোর দিন — সাইডবার লুকানো নিরাপত্তা নয়; এটা শুধু ভদ্রতা।

২০

পরের ক্লাস — "প্রমাণ দিয়ে পরিচয়"

অথেন্টিকেশন: লগইন · সেশন · পাসওয়ার্ড রিসেট · ইমেইল বনাম ইউজারনেম · ব্রুট-ফোর্স ও সেশন নিরাপত্তা

পর্ব ৪ · কোডিং · ক্লাস ২০ / ৩৭

প্রমাণ দিয়ে পরিচয়

সিলেবাস: প্র্যাকটিক্যাল ৬ · টপিক ৬.২ (অথেন্টিকেশন)

৯টি অ্যাকাউন্ট রুট · ৯টি টেস্ট · ৪টি পাসওয়ার্ড নিয়ম · ৩টি কাস্টম middleware

AbstractBaseUser · session · নিরাপত্তার স্তর

শুরু করুন প্রশ্ন দিয়ে

লগইনের পর আপনি কোথায় "মনে" থাকেন?

হাত তুলুন: ব্রাউজারে? সার্ভারে? ডেটাবেসে?

ব্রাউজারে থাকে শুধু একটি আইডি — আসল তথ্য সার্ভারের ডেটাবেসে।

তাই কেউ কুকি চুরি করলেও পাসওয়ার্ড পায় না, আর সার্ভার যেকোনো মুহূর্তে সেশন বাতিল করতে পারে। আজকের ক্লাসে এই প্রমাণ-ব্যবস্থার প্রতিটি স্তর — মডেল থেকে কুকি ফ্ল্যাগ পর্যন্ত।

শিক্ষকের নোট: ডিভ-টুলে `sessionid` কুকি দেখান, তারপর ডেটাবেসের `django_session` টেবিল। "এই দুটো মিলেই আপনি লগইন করা" — এটাই সবচেয়ে স্পষ্ট ব্যাখ্যা।

ধাপ ১

প্রথম সিদ্ধান্ত: কাস্টম User মডেল

প্রথম মাইগ্রেশনের **আগে** — নাহলে বদলানো যন্ত্রণার কাজ

```
class User(AbstractBaseUser, PermissionsMixin):
    email = models.EmailField(unique=True, db_index=True)
    full_name = models.CharField(max_length=150)
    phone = models.CharField(max_length=16, blank=True,
                             validators=[phone_validator])
    is_active = models.BooleanField(default=True)
    is_staff = models.BooleanField(default=False)
    date_joined = models.DateTimeField(default=timezone.now)

    objects = UserManager()
    USERNAME_FIELD = "email"
    REQUIRED_FIELDS = ["full_name"]
```

কেন ডিফল্ট User নয়

ডিফল্টে লগইন আইডি `username` — কিন্তু স্কুলের ব্যবহারকারী মনে রাখে ইমেইল।

ভূমিকা এখানে নেই

একজন মানুষ একাধিক প্রতিষ্ঠানে — তাই ভূমিকা `Membership`-এ, User-এ নয়।

মডেলের ৩টি ক্ষেত্র

Mobile / `full_name` / `is_active` + ডকুমেন্টে `last_login` ও পাসওয়ার্ড হ্যাশ।

দেরি করার খরচ

পরে বদলাতে ৭০টি ForeignKey-এর মাইগ্রেশন — তাই সিদ্ধান্ত দিন এক, দিন শূন্যতে।

শিক্ষকের নোট: IMS-এর মডেল ফাইলের শিরোনামেই লেখা: "never start a Django project with the default User model"। এটা মুখস্থ নিয়ম নয় — ৭০টি FK-এর হিসাব।

ধাপ ২

পাসওয়ার্ড কখনোই সংরক্ষণ নয়

`UserManager` — ইমেইল নরমালাইজ, তারপর `set_password()`

```
def _create_user(self, email, password, **extra):
    if not email:
        raise ValueError("An e-mail address is required.")
    email = self.normalize_email(email).lower()
    user = self.model(email=email, **extra)
    user.set_password(password)   # হ্যাশ করে, প্লেইন নয়
    user.save(using=self._db)
    return user

def create_superuser(self, email, password=None, **extra):
    extra.setdefault("is_staff", True)
    extra.setdefault("is_superuser", True)
    if extra.get("is_staff") is not True:
        raise ValueError("Superuser must have is_staff=True.")
    ...
```

কোন লাইন	কেন
`.lower()`	`Ayesha@Example.COM` → `ayesha@example.com`
`set_password`	PBKDF2 + সল্ট; DB-তে `pbkdf2_sha256$…`
`unique=True`	দুই অ্যাকাউন্ট, এক ইমেইল — অসম্ভব
`is_active=False`	অ্যাকাউন্ট মুছে না, শুধু লগইন বন্ধ

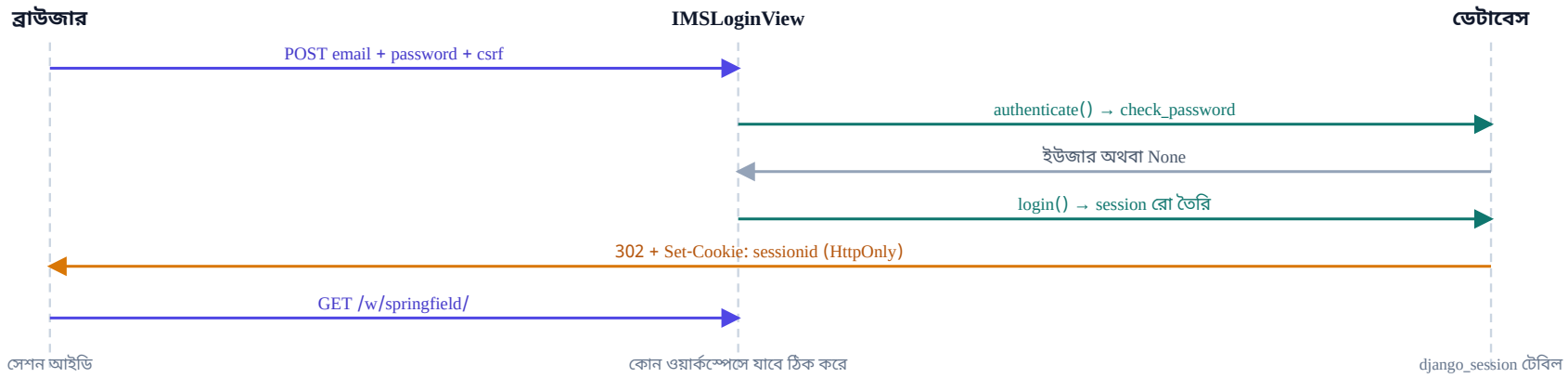
`create_superuser`-এ **স্পষ্ট** `ValueError` — ভুল কনফিগে চুপচাপ এগোয় না, সাথে সাথে থামে।

শিক্ষকের নোট: শেলে `user.password` প্রিন্ট করুন — লম্বা হ্যাশ দেখে ছাত্ররা বোঝে পাসওয়ার্ড আসলে কোথাও নেই। তারপর `check_password`।

ধাপ ৩

লগইনের সাত ধাপ

ফর্ম → `authenticate()` → `login()` → সেশন কুকি → রিডাইরেক্ট



`redirect_authenticated_user = True` — লগইন করা মানুষ লগইন পর্দা আর দেখেন না।

`form_valid`-এ পুরোনো `SESSION_KEY` মুছে দেওয়া হয় — নাহলে আগের প্রতিষ্ঠানেই ঢুকে পড়তেন।

শিক্ষকের নোট: দ্বিতীয় কার্ডটা বাস্তব বাগ থেকে: এক ব্রাউজারে দুই স্কুলের অ্যাডমিন লগইন করলে ভুল স্কুলে ঢুকে যাচ্ছিলেন। "সেশনেও পুরোনো স্মৃতি থাকে।"

ধাপ ৪

একই এরর, দুই কারণে

`IMSAuthenticationForm` — ইচ্ছাকৃতভাবে অস্পষ্ট

```
class IMSAuthenticationForm(AuthenticationForm):
    username = forms.EmailField(label="E-mail",
        widget=forms.EmailInput(attrs={
            "class": "form-control", "autofocus": True,
            "autocomplete": "email"}))
    password = forms.CharField(label="Password",
        widget=forms.PasswordInput(attrs={
            "class": "form-control",
            "autocomplete": "current-password"}))

    error_messages = {
        "invalid_login": "E-mail or password is incorrect.",
        "inactive": "This account is deactivated. "
                    "Contact your administrator.",
    }
```

পরিস্থিতি	ব্যবহারকারী যা দেখেন
ইমেইলই নেই	**"E-mail or password is incorrect."** দুই ক্ষেত্রেই হুবহু এক
ইমেইল আছে, পাসওয়ার্ড ভুল	
অ্যাকাউন্ট নিষ্ক্রিয়	আলাদা বার্তা (মালিককে জানানো দরকার)

কেন? আলাদা বার্তা মানে কেউ একা একা **কোন ইমেইলগুলো নিবন্ধিত** তা বের করে ফেলতে পারে।

`autocomplete`-এ `current-password` — পাসওয়ার্ড ম্যানেজার সঠিকভাবে কাজ করার শর্ত।

শিক্ষকের নোট: প্রশ্ন করুন: "নিষ্ক্রিয় অ্যাকাউন্টের কথা আলাদা করে বলা কি নিরাপদ?" উত্তর: কিছুটা তথ্য ফাঁস হয়, কিন্তু ব্যবহারকারীকে অকারণে দোষারোপ করা আরও খারাপ — এটা সচেতন সমঝোতা।

ধাপ ৫

সেশনের নিয়ম — কোডে লেখা আট লাইন

`config/settings/base.py` ও `prod.py`

সেটিং	মান	মানে
`SESSION_COOKIE_AGE`	৮ ঘণ্টা	এক কর্মদিবস — রাতে নিজেই শেষ
`SESSION_SAVE_EVERY_REQUEST`	True	প্রতিটি অনুরোধে মেয়াদ নবায়ন (কাজ করলে লগআউট নয়)
`EXPIRE_AT_BROWSER_CLOSE`	False	ব্রাউজার বন্ধ করলেই লগআউট হয় না
`SESSION_COOKIE_SECURE`	prod: True	শুধু HTTPS-এ কুকি যায়
`SESSION_COOKIE_HTTPONLY`	prod: True	JavaScript কুকি পড়তে পারে না
`SESSION_COOKIE_SAMESITE`	prod: Lax	অন্য সাইট থেকে পাঠানো POST-এ কুকি যায় না

লগ আউট = সেশন রো মুছে ফেলা + `SESSION_KEY` পপ। বাকি সব কপি তখন অর্থহীন।

এই তিনটি ফ্ল্যাগ **শুধু prod.py-তে** — লোকালে HTTP বলে চালু থাকলে লগইনই হতো না। পার্থক্যের কারণ জানুন।

শিক্ষকের নোট: বলুন: "৮ ঘণ্টা মানে শিক্ষকের এক কর্মদিবস।" সময়সীমা ব্যবসায়িক সিদ্ধান্ত, শুধু কারিগরি নয় — ব্যাংকে ১৫ মিনিট, এখানে ৮ ঘণ্টা।

ধাপ ৬

লগইনের পরে: কোন প্রতিষ্ঠানে?

`CurrentWorkspaceMiddleware` — প্রতিটি অনুরোধে একবার

```
ws_id = request.session.get(SESSION_KEY)   # active_workspace_id
qs = Membership.objects.filter(user=request.user, is_active=True)

if ws_id:
    m = qs.filter(workspace_id=ws_id, workspace__is_active=True).first()
    if m:
        return m

# না থাকলে: সর্বোচ্চ ভূমিকা, তারপর প্রতিষ্ঠান, তারপর নাম
m = (qs.filter(workspace__is_active=True)
       .order_by(ROLE_PRIORITY, KIND_PRIORITY, "workspace__name").first())
if m:
    request.session[SESSION_KEY] = m.workspace_id
return m
```

অগ্রাধিকার	কে আগে
`ROLE_PRIORITY`	ORG_ADMIN → COORDINATOR → TEACHER → STUDENT
`KIND_PRIORITY`	INSTITUTION আগে, PERSONAL পরে
শেষে	নাম অনুযায়ী — ফলাফল নির্ভরযোগ্য

বাস্তব বাগ: এখানে একবার **lazy** অবজেক্ট ফেরানো হয়েছিল। `lazy_obj is None` কখনো True হয় না — ওয়ার্কস্পেসহীন ব্যবহারকারী চেক পাশ করে গিয়েছিলেন।

শিক্ষকের নোট: লাইভে দেখান: এক ব্যবহারকারী দুই প্রতিষ্ঠানে — `bash run.sh` দিয়ে লগইন করে প্রথমে কোনটি খোলে, তারপর switch করে দেখান।

ধাপ ৭

রেজিস্ট্রেশন = তিনটি সারি, একসাথে

`transaction.atomic()` — অর্ধেক অ্যাকাউন্ট কখনো নয়

```
def form_valid(self, form):
    with transaction.atomic():
        user = form.save()
        workspace = Workspace.objects.create(
            kind=WorkspaceKinds.PERSONAL,
            name=form.cleaned_data["workspace_name"],
            owner=user)
        Membership.objects.create(
            user=user, workspace=workspace,
            role=Roles.TEACHER)
    login(self.request, user)
    self.request.session[SESSION_KEY] = workspace.pk
    return redirect("reports:dashboard", slug=workspace.slug)
```

টেস্ট	যা প্রমাণ করে
`test_registration_creates_user_workspace_and_membership`	৩ সারি + সেশন + ৩০২
`test_duplicate_email_is_rejected`	"already exists"
`test_password_mismatch_is_rejected`	"do not match"

টেস্টে ইমেইল `Ayesha@Example.COM` পাঠিয়ে যাচাই করা হয় `ayesha@example.com` সেভ হয়েছে কি না।

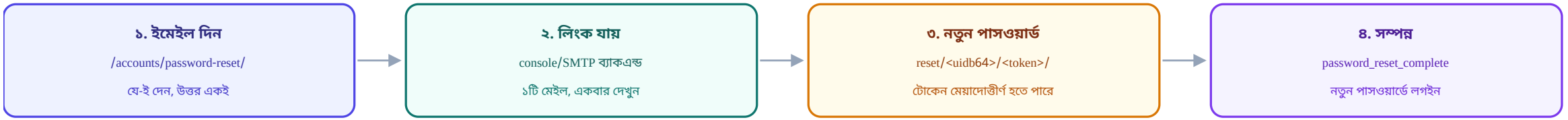
`atomic` না থাকলে দুর্ঘটনায় ব্যবহারকারী তৈরি হয়ে ওয়ার্কস্পেস ছাড়া থেকে যেতে পারে — "লগইন হয়, কিন্তু কোথাও নেই"।

শিক্ষকের নোট: সবুজ টেস্ট দেখান: `pytest apps/accounts` বা `manage.py test apps.accounts`। "দাবি নয়, প্রমাণ" — এই কোর্সের সুর।

ধাপ ৮

পাসওয়ার্ড ভুলে গেলে — চার ধাপ, দুই টোকেন

`uidb64` + `token` — লিংক একবারই কাজ করে, সময়সীমা আছে



টেস্ট: mail.outbox থেকে লিংক তুলে সত্যিকারের পথে হেঁটে নতুন পাসওয়ার্ড সেট করে যাচাই

অজানা ইমেইলেও **একই ৩০২ উত্তর** — কেউ অনুমান করে দেখতে পারে না কে নিবন্ধিত।

ডেমোতে ইমেইল কনসোলে ছাপা হয় (`EMAIL_BACKEND=console`) — প্রোডাকশনে SMTP।

টোকেন কেন দুটি

`uidb64` বলে কে, `token` বলে অনুমতি — শুধু আইডি জানা থাকলে লিংক বানানো যেত।

একবারই

পাসওয়ার্ড সেট হওয়ামাত্র টোকেন অকার্যকর — পুরোনো মেইল আবার ব্যবহার করা যায় না।

মেয়াদ

Django-র ডিফল্ট `PASSWORD_RESET_TIMEOUT` ৩ দিন — দীর্ঘ, কিন্তু সেটাও সচেতন পছন্দ।

শিক্ষকের নোট: `mail.outbox`-এর টেস্টটা দেখান: "মেইল সত্যিই গেছে কি না" — এটাই ওই ফিচারের একমাত্র নির্ভরযোগ্য প্রমাণ, কারণ UI কখনোই জানে না মেইল পৌঁছেছে কি না।

ধাপ ৯

পাসওয়ার্ড বদল — তিনটি ঘর, একটি কঠিন নিয়ম

`PasswordChangeForm` নিজেই পুরোনো পাসওয়ার্ড যাচাই করে

```
class IMSPasswordChangeView(PasswordChangeView):
    template_name = "accounts/password_change.html"
    form_class = IMSPasswordChangeForm
    success_url = reverse_lazy("accounts:profile")

    def form_valid(self, form):
        messages.success(self.request,
                         "Password changed successfully.")
        return super().form_valid(form)

class IMSPasswordChangeForm(PasswordChangeForm):
    old_password = forms.CharField(
        label="Current password",
        widget=forms.PasswordInput(attrs={
            "autocomplete": "current-password"}))
    new_password1 = forms.CharField(
        widget=forms.PasswordInput(attrs={
            "autocomplete": "new-password"}))
```

পুরোনো পাসওয়ার্ড কেন

কেউ চুরি করা সেশন নিয়ে পাসওয়ার্ড বদলে আপনাকে বের করে দিতে পারবে না।

একই নিয়ম আবার

নতুন পাসওয়ার্ডও ৪টি validator পার হতে হয় — ফর্ম বদলালেও নিয়ম বদলায় না।

সেশন টিকে থাকে

পাসওয়ার্ড বদলালে Django সেশন-কি ঘুরিয়ে দেয় — অন্য ডিভাইসের সেশন বাতিল হয় না, কিন্তু কুকি নতুন হয়।

প্রোফাইল আলাদা

`IMSUserProfileForm` — শুধু `full_name` ও `phone`; ইমেইল বদলানো এখানে নেই, ইচ্ছাকৃত।

শিক্ষকের নোট: প্রশ্ন করুন: "প্রোফাইল থেকে ইমেইল বদলানো গেলে কী ঝুঁকি?" উত্তর: লগইন আইডি বদলে যাওয়া + অন্য অ্যাকাউন্টের ইমেইলে দখল — তাই আলাদা, কঠিন প্রবাহ দরকার।

ধাপ ১০

লগইন পর্দাটাই প্রথম ছাপ

দুই কলাম · ব্র্যান্ডের নাম · কুকি গার্ড · ডেমো লগইন

```
{% extends "base.html" %}
<div class="ims-auth">            {# 1.05fr .95fr #}
  <aside class="ims-auth__aside">  {# বাম: ব্র্যান্ড #}
    {{ brand.name|default:"Teacher Management System" }}
    <h1>{{ brand.tagline|default:"Every class, every mark…" }}</h1>
  </aside>
  <main class="ims-auth__main">
    <h2>Sign in{% if brand.name and brand.name != product.short %}
        to {{ brand.name }}{% endif %}</h2>
    {% include "partials/cookie_guard.html" %}
    <form method="post" novalidate>
      {% csrf_token %}
      {% include "partials/field.html" with field=form.username %}
      {% include "partials/field.html" with field=form.password %}</form>
  </main></div>
```

ব্র্যান্ড বদলায় শিরোনাম

প্রতিষ্ঠানের নাম থাকলে "Sign in to Springfield High" — নাহলে শুধু "Sign in"।

কুকি গার্ড

১৫ লাইন JS পরীক্ষা করে কুকি লেখা যায় কি না — না গেলে টাইপ করার আগেই সতর্ক করে।

novalidate কেন

আসল যাচাই সার্ভারে; ব্রাউজারের বাধা শুধুই সুবিধা। এমবেড করা প্রিভিউতেও ফর্ম কাজ করে।

ডেমো লগইন বাক্স

`admin@ · teacher@ · student@ims.local` · `DemoPass!2026` — ক্লাসে সময় বাঁচায়।

শিক্ষকের নোট: কুকি ব্লক করা ব্রাউজারে (বা প্রাইভেট মোড + কঠোর সেটিং) লগইন পর্দা খুলে দেখান — সতর্কবার্তাটা নিজেই বলে দেয় সমস্যা কী।

ধাপ ১১

ছয়টি স্তর, তারপরও একটি ফাঁক

`prod.py`-তে যেগুলো আছে — আর যা এখনো নেই

স্তর	কী আটকায়
`CsrfViewMiddleware` (৩৬ ফর্মে টোকেন)	অন্য সাইট থেকে জাল POST
`SESSION_COOKIE_HTTPONLY`	XSS দিয়ে কুকি চুরি
`SECURE_SSL_REDIRECT` + HSTS ৩০ দিন	HTTP-তে পাসওয়ার্ড যাওয়া
`X_FRAME_OPTIONS = DENY`	ক্লিকজ্যাকিং
পাসওয়ার্ড হ্যাশ + ৪টি validator	দুর্বল পাসওয়ার্ড, ডাম্প/লিক হলে ক্ষতি
prod.py-এর `RuntimeError` গার্ড	ডিফল্ট SECRET_KEY বা `*` হোস্ট দিয়ে চালু হওয়া

যা এখনো নেই: লগইন প্রচেষ্টার সীমা (rate limit) · অ্যাকাউন্ট লকআউট · ইমেইল ভেরিফিকেশন · 2FA। কোডে `ratelimit` / `attempt` শব্দ দুটির খোঁজে **শূন্য** মিল — অর্থাৎ আজ কেউ ১০,০০০ বার পাসওয়ার্ড অনুমান করতে পারবে।

প্রতিকার সাধারণত ক্যাশ-ভিত্তিক কাউন্টার: `(ip, email)` কী-তে ৫ ব্যর্থতায় ১৫ মিনিট ব্লক — মডেল বদলাতে হয় না।

শিক্ষকের নোট: এই স্লাইডটা সৎ থাকুন: "প্রথম সংস্করণে এটাই স্বাভাবিক, কিন্তু প্রকাশের আগে জানা থাকতে হবে।" বাস্তব আক্রমণ: একটি স্কুলের ৪০০ ইমেইলে স্প্রে-হামলা।

অনুশীলন ও আলোচনা

২০ মিনিট — প্রমাণ নিজে পরীক্ষা করুন

তিনটি কাজ, তারপর চারটি প্রশ্ন

১. সেশন কোথায় (৬ মিনিট)

লগইন করে `sessionid` কুকি কপি করুন, তারপর `django_session` টেবিলে ওই কী খুঁজুন।

২. এরর বার্তা (৬ মিনিট)

অস্তিত্বহীন ইমেইল আর ভুল পাসওয়ার্ড — দুইবার লগইন করে দেখুন উত্তর হুবহু এক কি না।

৩. নিষ্ক্রিয় ব্যবহারকারী (৮ মিনিট)

`is_active=False` করে লগইন চেষ্টা করুন; তারপর `apps/accounts/tests.py`-এর টেস্টটি চালান।

- পাসওয়ার্ড কোথায় জমা থাকে — আর কী জমা থাকে না?
- একই এরর বার্তা কেন দুটি ভিন্ন কারণে?
- সেশন কুকি চুরি হলে পাসওয়ার্ডও যায় — সত্যি?
- rate limit না থাকলে প্রথম ২৪ ঘণ্টায় কী হবে?

শিক্ষকের নোট: তৃতীয় প্রশ্নের সঠিক উত্তর: "না — কুকি থাকলে ওই সেশনটাই ব্যবহার করা যায়, পাসওয়ার্ড নয়।" পার্থক্যটা ছাত্ররা প্রায়ই গুলিয়ে ফেলে।

সারমর্ম

ছয়টি কথা সঙ্গে নিন

প্রতিটির পাশে IMS-এর প্রমাণ

নীতি	প্রমাণ
কাস্টম User, প্রথম দিনেই	`USERNAME_FIELD = "email"` · ৭০টি FK
পাসওয়ার্ড কখনো প্লেইন নয়	`set_password()` · `pbkdf2_sha256$…`
এরর বার্তা তথ্য ফাঁস করে না	`invalid_login` এক, দুই কারণে
সেশন = সার্ভারে সত্য, ক্লায়েন্টে আইডি	`django_session` · HttpOnly · ৮ ঘণ্টা
লগইনের পরই ঠিক হয় কোন প্রতিষ্ঠান	`CurrentWorkspaceMiddleware` · ROLE_PRIORITY
যা লেখা হয়নি, তা সুরক্ষাও নয়	**rate limit শূন্য** · লকআউট নেই · 2FA নেই

শিক্ষকের নোট: শেষ সারিটা আবার বলুন: "নিরাপত্তার তালিকা তৈরি করা মানে নিজের ফাঁক জেনে রাখা।" পরের ক্লাসে ফাইল আপলোডেও একই ধরনের ফাঁক দেখবেন।

২১

পরের ক্লাস — "ফাইল ঢোকে, বিপদ ঢোকে না"

ফাইল আপলোড: `MEDIA_ROOT` · `upload_to` · ImageField · SVG/XSS ঝুঁকি · সাইজ ও ধরন যাচাই · ডাউনলোড অনুমতি

পর্ব ৪ · কোডিং · ক্লাস ২১ / ৩৭

ফাইল ঢোকে, বিপদ ঢোকে না

সিলেবাস: প্র্যাকটিক্যাল ৭ · টপিক ৭.২ (ফাইল আপলোড)

২টি ফাইল-ফিল্ড · ৯টি অনুমোদিত এক্সটেনশন · ১০ MB সীমা · ১টি ডাউনলোড ভিউ

MEDIA_ROOT · FileResponse · অনুমতি

শুরু করুন প্রশ্ন দিয়ে

আপলোড মানে তিনটি প্রশ্ন

ডিস্কে কোথায় · কে দেখতে পাবে · কোন ফাইল গ্রহণযোগ্য

প্রথমটি Django করে দেয়। দ্বিতীয়টি আপনার কাজ। তৃতীয়টি কেউ করে দেয় না।

IMS-এ ফাইল আপলোড হয় **এক জায়গায়** — শিক্ষকের লার্নিং ম্যাটেরিয়াল। কিন্তু ওই একটি ফিচারেই স্টোরেজ, অনুমতি, ভাইরাস-সদৃশ ঝুঁকি আর ব্যাকআপ — চারটি ভিন্ন সমস্যা একসাথে আসে।

শিক্ষকের নোট: বলুন: "আপলোড ফিচার মানে শুধু একটা বাটন নয় — এর পেছনে সিদ্ধান্তের সারি।" শিক্ষার্থীর জমা (Submission) শুধু টেক্সট — IMS ইচ্ছাকৃতভাবে ছাত্রদের ফাইল নেয় না।

ধাপ ১

ফাইল কোথায় থাকে — দুই ধরনের ফাইল

`MEDIA_ROOT = BASE_DIR / "media"` · `MEDIA_URL = "media/"`

```
# config/settings/base.py
MEDIA_URL = "media/"
MEDIA_ROOT = BASE_DIR / "media"

# config/urls.py — শুধু ডেভেলপমেন্টে
if settings.DEBUG:
    urlpatterns += static(settings.MEDIA_URL,
                          document_root=settings.MEDIA_ROOT)

# প্রোডাকশনে nginx এই কাজটি করে, Django নয়
location /media/ { alias /srv/ims/media/; }
```

`DEBUG=False` হলে `static()` কাজ করে না — তাই প্রোডাকশনে "/media/logo.png" ৪০৪।

ফাইল	পথ ও প্রবেশাধিকার
লোগো (`ImageField`)	`media/branding/logos/` — **সর্বজনীন**, লগইন পর্দায় দেখা যায়
ম্যাটেরিয়াল (`FileField`)	`media/materials/2026/11/` — **কখনো সরাসরি নয়**, শুধু ভিউ দিয়ে
স্ট্যাটিক (CSS/JS)	`static/` → `collectstatic` — ডিপ্লয়ের অংশ

লোগো আর পাঠ্য-ম্যাটেরিয়াল এক নয়: একটি ব্র্যান্ডিং, অন্যটি প্রতিষ্ঠানের গোপন সম্পদ। তাই দুই পথ।

শিক্ষকের নোট: ব্রাউজারে `/media/branding/logos/…` খুলে দেখান — লগইন ছাড়াই দেখা যায়। তারপর `/media/materials/…` চেষ্টা করুন: সরাসরি পথ কাজ করে, কিন্তু অ্যাপে সেটি ব্যবহার করাই হয় না।

ধাপ ২

`upload_to` — নামকরণই ভাগ্য নির্ধারণ করে

`materials/%Y/%m/` — তারিখ অনুযায়ী ভাঁজ, সংঘর্ষে Django নিজে নাম বদলায়

```python
class Material(TimeStamped):
    file = models.FileField(upload_to="materials/%Y/%m/")

# ব্যবহারকারী যা আপলোড করল
#   "Chapter 3 (final).pdf"
#
# ডিস্কে যা জমা হলো
#   media/materials/2026/11/Chapter 3 (final).pdf
#
# একই নাম আবার এলে
#   media/materials/2026/11/Chapter 3 (final)_aX9k2.pdf
#
# ডাউনলোডের সময় আসল নাম ফিরিয়ে দেওয়া হয়
return FileResponse(material.file.open("rb"),
                    as_attachment=True,
                    filename=material.file.name.split("/")[-1])
```

তারিখ-ভাঁজ কেন

এক ফোল্ডারে ১০,০০০ ফাইল থাকলে `ls`, ব্যাকআপ আর ডিস্ক-ইনোড দুটোই ভোগে।

সংঘর্ষ নিজেই সমাধান

`get_available_name()` — ৫০ জন একই "notes.pdf" দিলেও কিছু হারায় না।

নাম ব্যবহারকারীর হাতে

Django ফাইলনেম স্যানিটাইজ করে (`get_valid_name`), তাই `../../etc/passwd` লিখেও লাভ নেই।

কিন্তু...

নাম বদলানো মানে **ডাউনলোডে অদ্ভুত নাম** — তাই `filename=` স্পষ্ট দিতে হয় (উপরের কোড)।

শিক্ষকের নোট: লাইভে একই নামে দুইবার আপলোড করুন, তারপর `ls media/materials/` দেখান। ছাত্ররা অবাক হয় — "নাম তো বদলে গেল!" এই জ্ঞানটাই পরে ডাউনলোড বাগ ধরাতে সাহায্য করে।

ধাপ ৩

কোন ফাইল গ্রহণযোগ্য — মডেলেই লেখা

৯টি এক্সটেনশন · ১০ MB · `clean()`-এ যাচাই

```
class Material(TimeStamped):
    ALLOWED_EXTENSIONS = {"pdf", "docx", "xlsx", "pptx", "txt",
                          "jpg", "jpeg", "png", "zip"}
    MAX_BYTES = 10 * 1024 * 1024

    def clean(self):
        ext = (self.file.name.rsplit(".", 1)[-1].lower()
               if self.file else "")
        if ext and ext not in self.ALLOWED_EXTENSIONS:
            raise ValidationError(
                {"file": f"File type .{ext} is not allowed."})
        if self.file and hasattr(self.file, "size") \
                and self.file.size > self.MAX_BYTES:
            raise ValidationError(
                {"file": "File is larger than 10 MB."})
```

নিয়ম কোথায় থাকবে

মডেলে — কারণ সেটাই একমাত্র জায়গা যা **সব পথ** (ফর্ম, শেল, অ্যাডমিন, ম্যানেজমেন্ট কমান্ড) মানে।

`dict` ফেরানো

`ValidationError({"file": …})` — এরর ঠিক ফাইল ঘরে বসে, উপরে নয়।

সেট কেন

খোঁজ O(1); নতুন ধরন যোগ করা এক লাইনের কাজ — তালিকা পড়তেও সহজ।

সীমা

এটি শুধু **নামের শেষ অংশ** দেখে — ভেতরে কী আছে তা নয় (পরের স্লাইড)।

শিক্ষকের নোট: `.exe` বা `.html` আপলোড করে দেখান — ভদ্র এরর, ক্র্যাশ নয়। তারপর বলুন: "ভদ্র এররও একটি ফিচার।"

ধাপ ৪

যাচাই কে, কখন করে?

`MaterialForm.save(commit=False)` → `obj.clean()` → `obj.save()`

```
@workspace_view(*TEACHING)
def material_create(request, slug):
    form = MaterialForm(request.POST or None,
                        request.FILES or None,
                        workspace=request.workspace)
    if request.method == "POST" and form.is_valid():
        obj = form.save(commit=False)
        obj.workspace = request.workspace       # স্কোপ ভিউ থেকে
        obj.uploaded_by = request.user
        obj.clean()                             # আবার যাচাই
        obj.save()
        messages.success(request, "Material uploaded.")
        return redirect("lessons:material_list", slug=slug)
```

স্তর	কে ডাকে
`form.is_valid()`	ফিল্ড-ধরন, required, plus মডেলের `clean()`
`obj.clean()`	ভিউ নিজে — **workspace বসানোর পরে**
`obj.save()`	`clean()` ডাকে না — এটাই ফাঁদ

`request.FILES` না দিলে ফাইল কখনো আসে না — কিন্তু এররও আসে না, শুধু "This field is required"।

দ্বিতীয় `clean()` এখানে বাড়তি সতর্কতা — কারণ `workspace` সেট হওয়ার আগে মডেল-যাচাই চলেছিল।

শিক্ষকের নোট: `save()` নিজে যাচাই করে না — এটা মুখস্থ করার মতো নিয়ম: "save() বিশ্বাস করে, clean() যাচাই করে।" শেলে `Material(...).save()` চালিয়ে দেখান: বিশাল ফাইলও ঢুকে যায়।

ধাপ ৫

ডাউনলোড — IMS-এর সবচেয়ে গুরুত্বপূর্ণ ভিউ

"ফাইল কখনো সর্বজনীন URL থেকে দেওয়া হয় না"

```
@workspace_view(*ALL_ROLES)
def material_download(request, slug, pk):
    """Files are never served from a public URL."""
    material = get_object_or_404(Material, pk=pk,
                                 workspace=request.workspace)
    allowed = request.membership.role in (
        "ORG_ADMIN", "COORDINATOR", "TEACHER") or (
        teacher_can_access_class(request.user, material.class_group))
    student = getattr(request.user, "student_profile", None)
    if student:
        allowed = material.is_published and student.enrollments.filter(
            class_group=material.class_group, is_active=True).exists()
    if not allowed:
        raise PermissionDenied("You cannot download this file.")
    try:
        return FileResponse(material.file.open("rb"),
                            as_attachment=True,
                            filename=material.file.name.split("/")[-1])
    except FileNotFoundError:
        raise Http404("The file is missing on disk.")
```

কে	নিয়ম
শিক্ষক/কোঅর্ডিনেটর/অ্যাডমিন	নিজের স্কোপের ক্লাস হলে হ্যাঁ
ছাত্র	**প্রকাশিত** + নিজের ক্লাসে ভর্তি
অন্য প্রতিষ্ঠানের সদস্য	`get_object_or_404` → 404
লগইনহীন	decorator → লগইনে রিডাইরেক্ট

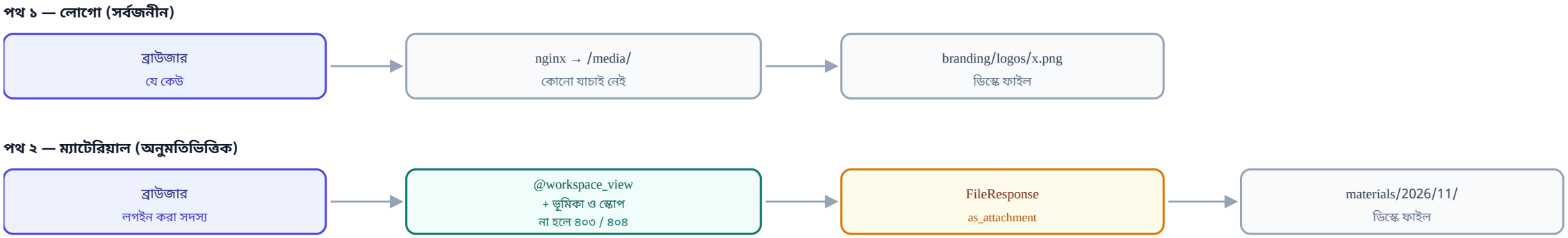
`as_attachment=True` — ব্রাউজারে খোলে না, ডাউনলোড হয়। HTML ফাইল খুলে গেলে XSS-এর দরজা।

শিক্ষকের নোট: ছাত্র হিসেবে লগইন করে অপ্রকাশিত ম্যাটেরিয়ালের URL হাতে চেষ্টা করুন — ৪০৩ আসবে। তারপর বলুন: "নিয়মটি UI-তে নয়, এখানে।"

ধাপ ৬

দুই পথ, এক প্রশ্ন: কে ঢুকতে পারে?

সর্বজনীন লোগো বনাম অনুমতিভিত্তিক ম্যাটেরিয়াল



নিয়ম: ব্যবহারকারীর দেওয়া যেকোনো ফাইল সবসময় পথ ২ দিয়ে যাবে — পথ ১ শুধু ব্র্যান্ডিং ছবির জন্য।

পরীক্ষা: ছাত্রের অ্যাকাউন্টে `/media/materials/…` সরাসরি খুললে ফাইল **দেখা যাবে** (nginx জানে না কে কে)। তাই অ্যাপে ওই URL কখনো তৈরি হয় না — এই সীমাটা জানা থাকা জরুরি।

শিক্ষকের নোট: এই স্লাইডটা সততার: "এখনো পর্যন্ত সুরক্ষা নির্ভর করে URL গোপন থাকার ওপর, যা নিরাপত্তা নয়।" প্রোডাকশনে করণীয়: media ফোল্ডার nginx-এ `internal` করে X-Accel-Redirect ব্যবহার।

ধাপ ৭

এক্সটেনশন যথেষ্ট নয়

তিন স্তরের যাচাই — নাম · ধরন · বিষয়বস্তু

স্তর	যা দেখা হয়	প্রতারণা
এক্সটেনশন (IMS-এ **আছে**)	`.pdf` `.docx` …	`report.pdf.exe` — শেষ অংশ `exe`, আটকাবে
MIME ধরন	`file.content_type`	ক্লায়েন্ট যা বলে, সেটাই — মিথ্যা বলা সহজ
ম্যাজিক বাইট (অনুপস্থিত)	ফাইলের প্রথম কয়েক বাইট	`%PDF` দিয়ে শুরু? — এটাই আসল প্রমাণ

SVG ভেতরে JavaScript ধারণ করতে পারে — IMS-এ SVG অনুমোদিত নয় (`ALLOWED_EXTENSIONS`-এ নেই), লোগোতেও ঝুঁকি (ক্লাস ১৮)।

Zip bomb: ১০ MB zip খুললে ১০ GB হতে পারে — `.zip` অনুমোদিত, কিন্তু সার্ভারে কেউ খোলে না, তাই ঝুঁকি কম।

ম্যাজিক বাইট যোগ করা যায় ছোটকিছু দিয়ে: প্রথম ৪ বাইট পড়ে `%PDF` কি না মিলিয়ে দেখা — `kind="magic"`।

শিক্ষকের নোট: লাইভ ডেমো: একটি `.txt` ফাইলের নাম বদলে `.pdf` করে আপলোড করুন — পাশ করবে। "এক্সটেনশন মিথ্যা বলতে পারে, তাই এর ওপর ভরসা সীমিত।"

ধাপ ৮

পাঁচটি ফাঁদ — আর প্রতিকার

প্রতিটি বাস্তব প্রজেক্টে দেখা যায়

ফাঁদ	যা ঘটে / প্রতিকার
১. `request.FILES` না দেওয়া	ফাইল সেভ হয় না, এররও আসে না → `MaterialForm(request.POST, request.FILES)`
২. ফাইল মুছলে ডিস্কে থেকে যায়	DB সারি গেলে ফাইল অনাথ → `delete()` override বা পরিষ্কার কমান্ড
৩. একই ফাইল দুবার আপলোড	ডিস্কে দুই কপি, ব্যাকআপ ফুলে ওঠে → হ্যাশ মিলিয়ে ডিডুপ্লিকেশন
৪. ওয়েব সার্ভারে `MEDIA_ROOT` নয়	ডেপ্লয়ে ফাইল যায়, কিন্তু ছবি ভাঙা → মিডিয়া ভলিউম/অবজেক্ট স্টোরেজ
৫. `DEBUG=False`-এ `static()` ভরসা	লোকালে চলে, সার্ভারে ৪০৪ → nginx অ্যালিয়াস বা S3

পাঁচটি ফাঁদের চারটিই "লোকালে কাজ করে, সার্ভারে ভাঙে" শ্রেণির। ডেপ্লয়ের আগে একবার সত্যিকারের সার্ভারে আপলোড → ডাউনলোড পরীক্ষা করুন।

শিক্ষকের নোট: দ্বিতীয় ফাঁদটা বোর্ডে আঁকুন: DB সারি মুছে গেল, অথচ ৩০০ MB PDF ডিস্কে পড়ে আছে। "ব্যাকআপ নিলে এই ভূতও ব্যাকআপ হয়।"

ধাপ ৯

ফাইল আপলোডের টেস্ট কীভাবে লিখবেন

`SimpleUploadedFile` — ব্রাউজার ছাড়াই

```
from django.core.files.uploadedfile import SimpleUploadedFile

def test_rejects_exe_and_accepts_pdf(self):
    self.client.force_login(self.teacher)
    url = reverse("lessons:material_create",
                  kwargs={"slug": self.ws.slug})

    bad = SimpleUploadedFile("notes.exe", b"MZ\x90\x00",
                             content_type="application/pdf")
    self.client.post(url, {"title": "X", "class_group": self.cls.pk,
                           "file": bad})
    self.assertFalse(Material.objects.exists())

    good = SimpleUploadedFile("notes.pdf", b"%PDF-1.4 ok",
                              content_type="application/pdf")
    self.client.post(url, {"title": "X", "class_group": self.cls.pk,
                           "file": good})
    self.assertEqual(Material.objects.count(), 1)
```

এখন IMS-এ কয়টি?

শূন্য — `apps/lessons/tests.py`-এ ৪টি টেস্ট, সবই হোমওয়ার্ক ও সিলেবাস নিয়ে; আপলোডের একটিও নেই।

কেন বিপজ্জনক

ডাউনলোডের অনুমতি-নিয়মটাই সবচেয়ে জটিল কোড, অথচ সেটাই অপরীক্ষিত।

যা লেখা উচিত

৩টি: বড় ফাইল প্রত্যাখ্যান · অপ্রকাশিত ফাইলে ছাত্রের ৪০৩ · অন্য প্রতিষ্ঠানের ৪০৪।

মিডিয়া ফোল্ডার

টেস্ট চলাকালে `MEDIA_ROOT` অস্থায়ী ডিরেক্টরিতে দিন — নাহলে টেস্ট ফাইল রিপোতে জমে।

শিক্ষকের নোট: "যেটা টেস্ট করা হয়নি, সেটা কাজ করে — এই বিশ্বাসটাই ভাঙে।" ৫ মিনিটে ওই তিনটি টেস্ট নিজে লিখিয়ে নিন; এটাই সবচেয়ে মূল্যবান অনুশীলন।

ধাপ ১০

৩০০ MB ভিডিও পাঠালে কী হয়?

পাঁচটি সীমা, পাঁচটি ভিন্ন এরর — ব্যবহারকারী শুধু "ব্যর্থ" দেখেন

স্তর	ডিফল্ট	ফলাফল
nginx `client_max_body_size`	১ MB	**৪১৩** — Django পর্যন্ত পৌঁছায়ই না
`DATA_UPLOAD_MAX_MEMORY_SIZE`	২.৫ MB	POST ডেটা বড় হলে `RequestDataTooBig`
`FILE_UPLOAD_MAX_MEMORY_SIZE`	২.৫ MB	এর বেশি হলে RAM-এর বদলে টেম্প ফাইলে
`Material.MAX_BYTES`	১০ MB	ভদ্র এরর: "File is larger than 10 MB."
ডিস্ক	—	জমা হলে ব্যাকআপ সময় ও জায়গা দুটোই বাড়ে

nginx-এর সীমা ডিফল্ট ১ MB — তাই ১০ MB-র নিয়ম থাকলেও ব্যবহারকারী **পাঁচ MB PDF-তে ৪১৩** পাবেন, যদি `client_max_body_size 20m;` না লেখা হয়।

সীমাগুলো **একই সংখ্যায়** মিলিয়ে দিন (nginx ২০ MB ⊃ অ্যাপ ১০ MB) — নাহলে দুটি ভিন্ন বার্তায় ব্যবহারকারী বিভ্রান্ত।

শিক্ষকের নোট: "ভিডিও আপলোড চাইলে উত্তর 'না' — কারণ ৩০০ MB × ৪০০ শিক্ষক = প্রতি মাসে Terra-bytes।" আপলোড নীতি প্রায়ই স্টোরেজ-নীতি, শুধু কোড নয়।

ধাপ ১১

মিডিয়া ব্যাকআপের একটি নীরব ফাঁদ

`ops/backup.sh` — DB ডাম্প + মিডিয়া টার, ১৪ দিনের রিটেনশন

```
#!/usr/bin/env bash
set -euo pipefail
STAMP=$(date +%Y%m%d-%H%M%S)
DEST="${BACKUP_DIR:-./backups}"
mkdir -p "$DEST"
pg_dump -Fc -f "$DEST/ims-$STAMP.dump"
tar -czf "$DEST/media-$STAMP.tar.gz" media/ 2>/dev/null || true
find "$DEST" -name '*.dump' -mtime +14 -delete
find "$DEST" -name '*.tar.gz' -mtime +14 -delete
```

`|| true` — মিডিয়া টার ব্যর্থ হলেও স্ক্রিপ্ট **সফল** দেখায়। ব্যাকআপ আছে ভেবে দিন, অথচ ফাইলগুলো নেই।

DB + ফাইল একসাথে

শুধু `pg_dump` নিলে সারি ফিরে আসে, ফাইল ফেরে না — সব ডাউনলোড ৪০৪।

১৪ দিন কেন

দুই সপ্তাহের বেশি পুরোনো ব্যাকআপ কেউ খোলে না — কিন্তু ভুল ধরা পড়ে ৩ দিনে।

প্রতিকার

`|| true` বাদ → ব্যর্থ হলে স্ক্রিপ্ট **থামুক**; নাহলে মনিটরিং কখনো জানবে না।

ImageField আলাদা

Pillow দরকার (requirements-এ আছে); উচ্চতা/প্রস্থ পড়তে চাইলে `width_field`।

শিক্ষকের নোট: প্রশ্ন করুন: "৫০০ MB মিডিয়া, ১ KB ডেটাবেস — কোনটা বেশি ঝুঁকিতে?" উত্তর: মিডিয়া, কারণ সেটার কোনো ট্রানজ্যাকশন-লগ নেই।

অনুশীলন ও আলোচনা

২০ মিনিট — এক ফিচার, চার পরীক্ষা

নিজের প্রজেক্টে একটি আপলোড যোগ করুন

১. মডেল (৫ মিনিট)

`FileField(upload_to="files/%Y/%m/")` + `ALLOWED_EXTENSIONS` + `clean()`।

২. ভিউ (৭ মিনিট)

ডাউনলোড ভিউ: `get_object_or_404(..., workspace=…)` + ভূমিকা যাচাই + `FileResponse`।

৩. প্রত্যাখ্যান (৫ মিনিট)

১১ MB ফাইল ও `.exe` — দুটোই আটকাচ্ছে কি না দেখুন।

৪. টেস্ট (৩ মিনিট)

`SimpleUploadedFile` দিয়ে গ্রহণ + প্রত্যাখ্যান, দুটোই।

- লোগো ও পাঠ্য-ফাইল — দুই পথ কেন?
- এক্সটেনশন যাচাই কেন অসম্পূর্ণ?
- `save()` কেন `clean()` ডাকে না?
- ফাইল ডিলিট হলে ডিস্কে কী থাকে?

শিক্ষকের নোট: তৃতীয় প্রশ্নের উত্তর: "কারণ `save()` কে ডাকছে তা জানে না — শেল, মাইগ্রেশন, কমান্ডও ডাকে।" তাই যাচাই সবসময় আলাদা স্তরে।

সারমর্ম

ছয়টি কথা সঙ্গে নিন

প্রতিটির পাশে IMS-এর প্রমাণ

নীতি	প্রমাণ
ফাইল ডিস্কে, DB-তে শুধু পথ	`MEDIA_ROOT` · `FileField` নাম রাখে
`upload_to`**-এ তারিখের ভাঁজ**	`materials/%Y/%m/` · `branding/logos/`
নিয়ম মডেলে, বার্তা ফর্মে	`ALLOWED_EXTENSIONS` · `MAX_BYTES` · `clean()`
ডাউনলোড সবসময় অনুমতি-যাচাই করা	`material_download` · `FileResponse`
এক্সটেনশন ≠ প্রমাণ	ম্যাজিক বাইট যাচাই এখনো **নেই**
অপরীক্ষিত পথ = অজানা পথ	আপলোড টেস্ট **০টি** · ডাউনলোড নিয়ম অপরীক্ষিত

শিক্ষকের নোট: শেষ দুই সারিতেই আজকের আসল শিক্ষা: "যা করিনি, তা লিখে রাখুন" — শুধু গর্বের তালিকা নয়, ফাঁকের তালিকাও।

২২

পরের ক্লাস — "সংখ্যা থেকে সিদ্ধান্ত"

রিপোর্ট ও এক্সপোর্ট: `annotate`/`aggregate` · রেজাল্ট শিট · CSV-তে BOM · pagination · বড় ডেটাসেট

পর্ব ৪ · কোডিং · ক্লাস ২২ / ৩৭

সংখ্যা থেকে সিদ্ধান্ত

সিলেবাস: প্র্যাকটিক্যাল ৭ · টপিক ৭.৩ (রিপোর্ট ও এক্সপোর্ট)
৭টি রিপোর্ট রুট · ৩টি CSV · ১টি PDF · ১১টি অ্যাসেসমেন্ট টেস্ট

aggregate · Decimal · BOM · marksheet

শুরু করুন প্রশ্ন দিয়ে

রিপোর্টে সবচেয়ে কঠিন অংশ কোনটি?

হাত তুলুন: কুয়েরি? চার্ট? PDF? নাকি অনুমতি?

উত্তর: হিসাব আর অনুমতি — কুয়েরি Django শিখিয়ে দেয়, কিন্তু "কার ফল দেখা যাবে" সেটা কেউ বলে দেয় না।

আজকের পথ: একটি নম্বর → শতাংশ (Decimal) → গ্রেড (band lookup) → সারসংক্ষেপ → CSV/PDF। প্রতিটি ধাপে একটি করে সিদ্ধান্ত, আর প্রতিটির পাশে টেস্ট।

শিক্ষকের নোট: বলুন: "রিপোর্টে বাগ মানে ভুল নম্বর, আর ভুল নম্বর অভিভাবককে ফোন করেন।" তাই এখানে টেস্ট সবচেয়ে বেশি দরকার — অথচ `apps/reports/tests.py` **নেই**; হিসাব টেস্ট করা আছে assessments-এ।

ধাপ ১

১১টি টেস্ট — হিসাবের প্রমাণ

`apps/assessments/tests.py` — প্রতিটি নিয়মের পাশে একটি টেস্ট

টেস্ট	যা রক্ষা করে
`test_grade_band_lookup`	শতাংশ → গ্রেড
`test_overlapping_bands_are_rejected`	দুই ব্যান্ড একই নম্বরে নয়
`test_result_sheet_handles_absent_zero_and_boundaries`	অনুপস্থিত · শূন্য · সীমানা
`test_marks_entry_rejects_out_of_range_value`	পূর্ণমানের বেশি নয়
`test_results_are_invisible_until_published`	প্রকাশের আগে লুকানো
`test_student_cannot_see_another_students_marksheet`	নিজের ফল ছাড়া নয়
`test_marksheet_pdf_is_rendered`	PDF সত্যিই তৈরি হয়

এই সাতটি দেখলেই বোঝা যায় —**রিপোর্ট মানে শুধু যোগফল নয়**; সীমা, অনুমতি আর প্রকাশের সময়ও হিসাবের অংশ।

`apps/reports/`-এ টেস্ট **০** — CSV-র কলাম ক্রম, ফাইলের নাম, BOM — কিছুই অপরীক্ষিত।

শিক্ষকের নোট: টেস্টের নাম পড়েই ছাত্ররা অনেক সময় ধারণা পেয়ে যায় কোডে কী আছে। বলুন: "ভালো টেস্টের নাম একটা বাক্যের ডকুমেন্টেশন।"

ধাপ ২

নম্বর থেকে গ্রেড — একটাই ফাংশন

`grade_for(percent, scale)` — সব পর্দা, CSV ও PDF একই ফাংশন ডাকে

```
def grade_for(percent, scale):
    """Resolve a percentage to (letter, gpa)."""
    if scale is None or percent is None:
        return ("-", Decimal("0.00"))
    p = Decimal(str(percent))
    for band in scale.bands.all():        # ordering = -min_percent
        if band.min_percent <= p <= band.max_percent:
            return (band.letter, band.gpa)
    return ("-", Decimal("0.00"))

class GradeBand(models.Model):
    letter = models.CharField(max_length=5)
    min_percent = models.DecimalField(max_digits=5, decimal_places=2)
    max_percent = models.DecimalField(max_digits=5, decimal_places=2)
    gpa = models.DecimalField(max_digits=3, decimal_places=2)
```

সিদ্ধান্ত	কারণ
এক ফাংশন, তিন ভোক্তা	স্ক্রিন · CSV · PDF-এ গ্রেড কখনো আলাদা হয় না
`Decimal(str(p))`	`75.0` → `"75.0"` — বাইনারি ভুল এড়ায়
ব্যান্ড না মিললে `"-"`	ক্র্যাশ নয়; ফাঁকা গ্রেড দেখানো
একই workspace-এ ১টি active scale	partial unique constraint (ক্লাস ১৫)
ওভারল্যাপ নিষিদ্ধ	নাহলে কোন ব্যান্ড আগে মিলবে তা অনিশ্চিত

শিক্ষকের নোট: "দুই ব্যান্ড একই নম্বরে মিললে কী হয়?" — প্রথমটা জেতে, মানে ডেটার ক্রম ফলাফল বদলায়। ওভারল্যাপ-টেস্ট এটাই আটকায়।

ধাপ ৩

রেজাল্ট শিট — বিশুদ্ধ ফাংশন

`build_result_sheet(exam)` — কোনো ডেটাবেস লেখা নেই, তাই টেস্ট করা সহজ

```
for mark in exam.marks.select_related("student"):
    obtained = None if mark.is_absent else mark.obtained
    percent = None
    if obtained is not None and exam.full_marks:
        percent = (Decimal(obtained) * 100 / Decimal(exam.full_marks)
                   ).quantize(Decimal("0.01"), rounding=ROUND_HALF_UP)
    letter, gpa = grade_for(percent, scale)
    rows.append({"student": mark.student, "obtained": obtained,
                 "percent": percent, "letter": letter, "gpa": gpa,
                 "is_pass": obtained is not None
                            and obtained >= exam.pass_marks})

scored = [r for r in rows if r["obtained"] is not None]
summary = {"students": len(rows), "scored": len(scored),
           "absent": ..., "passed": ..., "failed": ...,
           "pass_rate": round(passed * 100 / total_scored, 1),
           "average": ..., "highest": ..., "lowest": ...}
```

অনুপস্থিত আলাদা

`obtained=None` — শূন্য নয়! নাহলে গড় আর পাশের হার দুটোই ভুল হবে।

`scored` তালিকা

গড়-পাশ কেবল যারা পরীক্ষা দিয়েছে তাদের নিয়ে — `total_scored`।

ডেটাবেস লেখে না

পিওর ফাংশন মানে একই ইনপুটে একই আউটপুট — টেস্টে `self.assertEqual` দিয়ে ধরা যায়।

শূন্য দিয়ে ভাগ

কেউ পরীক্ষা দেয়নি? `if total_scored else 0` — নাহলে `ZeroDivisionError`।

শিক্ষকের নোট: বোর্ডে হিসাব করুন: ১০ জনের মধ্যে ২ জন অনুপস্থিত, বাকি ৮ জনের গড় ৬০ — "ক্লাসের গড় ৬০ নাকি ৪৮?" এই আলোচনাই রিপোর্টের রাজনীতি।

ধাপ ৪

`Decimal`, `float` নয়

টাকার হিসাব আর শতাংশে বাইনারি ভুল মেনে নেওয়া যায় না

```
# float — যা ঘটে
>>> 0.1 + 0.2
0.30000000000000004
>>> round(2.675, 2)
2.67          # প্রত্যাশা 2.68

# Decimal — যা চাই
Decimal("2.675").quantize(Decimal("0.01"),
                          rounding=ROUND_HALF_UP)   # 2.68

# IMS-এ একই নিয়ম
percent = (Decimal(obtained) * Decimal("100")
           / Decimal(exam.full_marks)).quantize(
               Decimal("0.01"), rounding=ROUND_HALF_UP)
```

কোথায় Decimal

`GradeBand.min/max/gpa` · ইনভয়েসের `price/amount` · শতাংশ

`ROUND_HALF_UP` কেন

Python-এর ডিফল্ট banker's rounding (`2.5 → 2`) — স্কুলের নম্বরে অপ্রত্যাশিত।

সীমানার ঝুঁকি

৩২.৯৯% vs ৩৩% — পাশ/ফেল ঠিক এই জায়গায় নির্ধারিত হয়, তাই `quantize` আগে, তুলনা পরে।

CSV-তে

`Decimal` সরাসরি লেখা যায় না → `str(percent)` ; নাহলে `csv` মডিউল TypeError দেয়।

শিক্ষকের নোট: টার্মিনালে `0.1+0.2` চালিয়ে দেখান — ক্লাসে সবসময় একই "ওহ!" আসে। তারপর বলুন: "ব্যাংকে ১ পয়সা, এখানে ১ নম্বর।"

ধাপ ৫

CSV-র চারটি লাইন যা সবাই ভুলে যায়

`_csv_response()` — এক ফাংশন, তিনটি এক্সপোর্ট

```
def _csv_response(filename, header, rows):
    resp = HttpResponse(content_type="text/csv; charset=utf-8")
    resp["Content-Disposition"] = (
        f'attachment; filename="{filename}"')
    resp.write("\ufeff")          # BOM so Excel opens UTF-8
    writer = csv.writer(resp)
    writer.writerow(header)
    for row in rows:
        writer.writerow(row)
    return resp
```

লাইন	না থাকলে
`charset=utf-8`	বাংলা নাম মুজিবকে দেখায়
`\ufeff` (BOM)	Excel-এ বাংলা অক্ষর ভাঙে — শুধু Excel-এ, LibreOffice-এ নয়
`attachment`	ব্রাউজারে কাঁচা টেক্সট খুলে যায়
`csv.writer`	নামে কমা থাকলে কলাম সরে যায় (`"Rahman, Jr."`)

ফাইলের নামেও তথ্য: `attendance-Class-8-2026-10-06-to-2026-11-05.csv` — খুলেই বোঝা যায় কী।

শিক্ষকের নোট: BOM-টা চোখে দেখা যায় না, তাই লাইভে Excel-এ খুলে দেখান। "যে কোডটা দেখা যায় না, সেটাই সবচেয়ে বেশি ভুল হয়।"

ধাপ ৬

তিনটি এক্সপোর্ট, তিনটি স্কোপ

একই `_students_for()` / `_classes_for()` — স্ক্রিনে যা, CSV-তে তাই

ফাইল	সোর্স	কলাম ও নিয়ম
`students.csv`	`_students_for()` + `q` / `status` ফিল্টার	ভর্তি নং · নাম · ক্লাস · অভিভাবক · ফোন · স্ট্যাটাস
`attendance-…csv`	`_classes_for()` + ৩০ দিনের ডিফল্ট রেঞ্জ	মোট · উপস্থিত · অনুপস্থিত · শতাংশ
`results-{exam}.csv`	`build_result_sheet(exam)`	নম্বর · পূর্ণমান · % · গ্রেড · পাশ/ফেল

তিনটিই `@workspace_view(*TEACHING)` — ছাত্র CSV নামাতে পারেন না। UI-তে বাটন নেই, সার্ভারেও নিয়ম আছে।

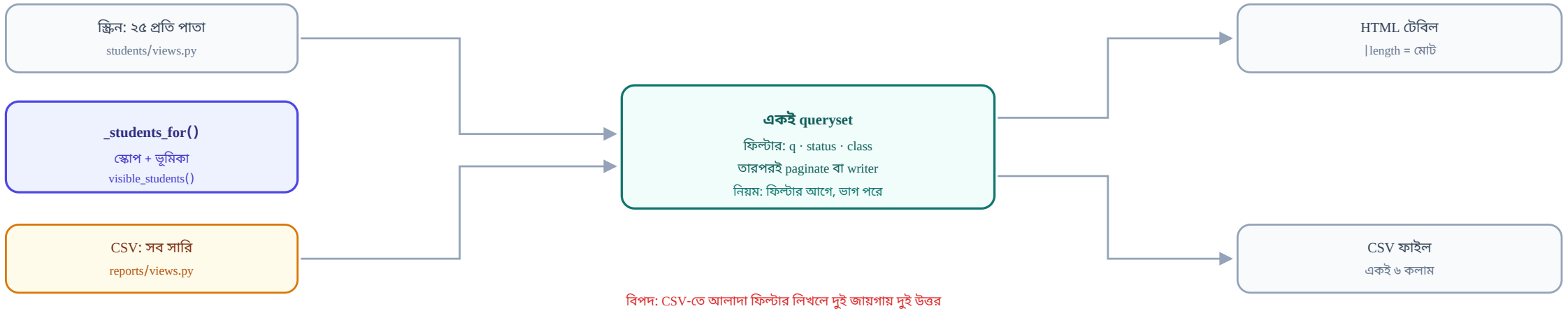
`export_results`-এ আলাদা করে `if role == "STUDENT": raise PermissionDenied` — বাড়তি সতর্কতা, কারণ `TEACHING`-এ STUDENT নেই।

শিক্ষকের নোট: "রিপোর্ট = আরেকটি দরজা" — এই বাক্যটাই ক্লাসের কেন্দ্র। প্রতিটি এক্সপোর্ট নতুন করে অনুমতি লিখলে একদিন কেউ ভুলে যাবে; তাই একই helper।

ধাপ ৭

স্ক্রিন আর CSV — একই পথ

দুই আলাদা কুয়েরি মানে দুই রকম সত্য



স্ক্রিনে ২৫টি দেখালেও CSV-তে **সব** যায় — কিন্তু দুটোই একই কুয়েরি থেকে, তাই "স্ক্রিনে যা দেখিনি CSV-তে তা এল" এমন বিভ্রান্তি হয় না।

শিক্ষকের নোট: প্রশ্ন করুন: "CSV-তে pagination থাকলে কী হয়?" উত্তর: ব্যবহারকারী প্রথম ২৫ জনের রিপোর্ট ভেবে বাকিদের বাদ দিয়ে ফেলেন — বিপজ্জনক নীরব ভুল।

ধাপ ৮

শতাংশের হিসাব ডেটাবেসেই

শর্তসহ `Count` — ৪০ জনের জন্য ৪০টি কুয়েরি নয়

```
agg = (AttendanceRecord.objects
       .filter(class_group=cls, date__range=(start, end))
       .values("student__admission_no", "student__full_name")
       .annotate(
           total=Count("id"),
           present=Count("id", filter=Q(status__in=["PRESENT", "LATE"])),
           absent=Count("id", filter=Q(status="ABSENT")))
       .order_by("student__full_name"))

rows = [(a["student__admission_no"], a["student__full_name"],
         a["total"], a["present"], a["absent"],
         round(a["present"] * 100 / a["total"]) if a["total"] else 0)
        for a in agg]
```

`values()` আগে

গ্রুপিং হয় ছাত্র ধরে; না দিলে পুরো ক্লাসের একটাই সারি আসবে।

`filter=Q(…)`

তিনটি সংখ্যা এক কুয়েরিতে — উপস্থিত, অনুপস্থিত, মোট।

শতাংশ Python-এ

SQL-এ ভাগ করলে `ZeroDivisionError` বা পূর্ণসংখ্যার ভাগ — তাই শেষ ধাপে, `if total else 0` দিয়ে।

বিকল্প

`annotate` ছাড়া প্রতিটি ছাত্রে ৩টি কুয়েরি = ১২০ কুয়েরি (N+1)।

শিক্ষকের নোট: `django.db.connection.queries` চালিয়ে দুই সংস্করণের কুয়েরি সংখ্যা দেখান — ১ বনাম ১২০। সংখ্যাটাই এখানে যুক্তি।

ধাপ ৯

রিপোর্টের ফিল্টার = ব্যবহারকারীর প্রশ্ন

ডিফল্ট মান লিখে রাখা মানে ব্যবহারকারীর হয়ে ভেবে নেওয়া

```
q = (request.GET.get("q") or "").strip()
if q:
    qs = qs.filter(full_name__icontains=q)
if request.GET.get("status"):
    qs = qs.filter(status=request.GET["status"])

today = timezone.localdate()
start = (request.GET.get("from")
         or (today - timedelta(days=30)).isoformat())
end = request.GET.get("to") or today.isoformat()
filter(class_group=cls, date__range=(start, end))

cls = (_classes_for(request)
       .filter(pk=request.GET.get("class")).first()
       or _classes_for(request).first())
```

নীতি	কারণ
ডিফল্ট ৩০ দিন	খালি রিপোর্টের চেয়ে "গত মাস" বেশি কাজে লাগে
`timezone.localdate()`	সার্ভার UTC, স্কুল ঢাকা — তারিখ গড়মিল এড়ায়
`?class=` না মিললে প্রথম ক্লাস	কখনো ভাঙা পর্দা নয়, তবে **ভুল ক্লাসও হতে পারে**
ফিল্টার আগে, ভাগ পরে	নাহলে টোটাল আর সারি মেলে না

`?class=999` (অন্যের ক্লাস) দিলে `_classes_for()`-এ না মিলে **প্রথম ক্লাস** দেখাবে — ৪০৩ নয়। ব্যবহারকারী বিভ্রান্ত হতে পারেন।

শিক্ষকের নোট: শেষ কার্ডটা নিয়ে আলোচনা করুন: নীরব ফলব্যাক ভালো UX, কিন্তু ভুল রিপোর্টের ঝুঁকি। বিকল্প: "এই ক্লাসে আপনার প্রবেশাধিকার নেই" বার্তা।

ধাপ ১০

মার্কশিট — একই ডেটা, দুই চেহারা

HTML পর্দা · PDF (`reportlab`) · অনুমতি একই

```
@workspace_view(*ALL_ROLES)
def marksheet(request, slug, pk):
    exam = get_object_or_404(Exam, pk=pk, workspace=request.workspace)
    student = getattr(request.user, "student_profile", None)
    if student:
        if not exam.is_published:
            raise PermissionDenied("Results are not published yet.")
        mark = exam.marks.filter(student=student).first()
        if mark is None:
            raise PermissionDenied("You have no result for this exam.")
        return _render_sheet(request, exam, [mark], own=True)
    if not teacher_can_access_class(request.user, exam.class_group):
        raise PermissionDenied("You are not assigned to this class.")
    return _render_sheet(request, exam, exam.marks..., own=False)

# ?format=pdf → render_marksheet_pdf(...)
resp["Content-Disposition"] = 'inline; filename="marksheet-3.pdf"'
```

এক ভিউ, দুই ফরম্যাট

`?format=pdf` — একই অনুমতি, একই সারি; শুধু আউটপুট বদলায়।

`inline` বনাম `attachment`

মার্কশিট ব্রাউজারে খোলা দরকার (প্রিন্ট), তাই `inline` ; CSV নামা দরকার, তাই `attachment` ।

শিক্ষকের নিয়ম

শুধু নিজের ক্লাসের — `teacher_can_access_class()` , নাহলে ৪০৩।

ছাত্রের নিয়ম

প্রকাশিত হতে হবে + নিজের সারি। দুই শর্তের একটি বাদ পড়লে ফল ফাঁস।

PDF লাইব্রেরি

`reportlab` — টেবিল নিজে আঁকে (`TableStyle`), হেডার প্রতি পাতায় (`repeatRows=1`)।

শিক্ষকের নোট: HTML বনাম PDF-এর পার্থক্য দেখান: ব্রাউজারের Ctrl+P-ও কাজ করে, কিন্তু প্রতিষ্ঠানের হেডার-লোগোসহ PDF আলাদা পণ্য — অভিভাবককে পাঠানোর জিনিস।

সতর্কতা

রিপোর্টের পাঁচটি ফাঁদ

প্রতিটির একটি করে বাস্তব চিহ্ন

ফাঁদ	চিহ্ন ও প্রতিকার
১. অনুপস্থিতকে শূন্য গোনা	গড় কমে যায় → `obtained=None` রাখুন, আলাদা গোনা
২. `float` দিয়ে শতাংশ	৩২.৯৯ / ৩৩ সীমানায় দোলাচল → `Decimal` + `quantize`
৩. CSV-তে pagination	অর্ধেক রিপোর্ট নীরবে যায় → রপ্তানিতে সব সারি
৪. মৃত কোড রিপোর্টে	`class_summary()`-তে `for … if False else []` — কখনো চলে না, পড়তে বিভ্রান্তিকর
৫. এক্সপোর্টে আলাদা অনুমতি	নতুন রুটে স্কোপ ভুলে যাওয়া → একই `_students_for()`

চতুর্থটি দেখতে ছোট, কিন্তু ভয়ংকর: পরের ডেভেলপার ভাববেন ওই লুপ কিছু একটা করে — আর সেটাকে "ঠিক" করতে গিয়ে নতুন বাগ আনবেন।

শিক্ষকের নোট: চতুর্থটা লাইভে খুলে দেখান — `if False else []` লেখাটা কেউ বিশ্বাসই করবে না। শিক্ষা: "মৃত কোড মুছে ফেলা একটা ফিচার।"

অনুশীলন ও আলোচনা

২০ মিনিট — একটি রিপোর্ট, চার শর্ত

নিজের প্রজেক্টে একটি CSV রিপোর্ট

১. স্কোপ (৫ মিনিট)

একই helper থেকে কুয়েরি নিন — নতুন ফিল্টার লিখবেন না।

২. শর্তসহ গণনা (৮ মিনিট)

`values()` + `annotate(Count(..., filter=Q(...)))` — এক কুয়েরিতে ২টি সংখ্যা।

৩. CSV (৫ মিনিট)

BOM + `charset` + `attachment`; বাংলা নাম দিয়ে Excel-এ খুলে যাচাই।

৪. সীমানা (২ মিনিট)

০ সারি ও ০ দিয়ে ভাগ — দুটোই পরীক্ষা করুন।

- অনুপস্থিত ছাত্র গড়ে ঢুকলে কী ভুল হয়?
- CSV-তে BOM কেন দরকার?
- রিপোর্টে আলাদা অনুমতি লিখলে কী ঝুঁকি?
- PDF আর HTML — কখন কোনটি?

শিক্ষকের নোট: চতুর্থ প্রশ্নের ভালো উত্তর: "PDF হলো যা পাঠানো যায়, HTML হলো যা ব্যবহার করা যায়।" অভিভাবকের কাছে PDF, শিক্ষকের কাছে পর্দা।

সারমর্ম

ছয়টি কথা সঙ্গে নিন

প্রতিটির পাশে IMS-এর প্রমাণ

নীতি	প্রমাণ
হিসাব একটি বিশুদ্ধ ফাংশনে	`build_result_sheet()` · `grade_for()`
অনুপস্থিত ≠ শূন্য	`obtained=None` · `scored` তালিকা
টাকায় ও নম্বরে `Decimal`	`quantize(…, ROUND_HALF_UP)`
CSV = BOM + charset + attachment	`_csv_response()` · ৩টি এক্সপোর্ট
শর্তসহ গণনা এক কুয়েরিতে	`Count("id", filter=Q(…))`
স্ক্রিন ও রপ্তানি একই অনুমতি	`_students_for()` / `_classes_for()`

শিক্ষকের নোট: প্রথম ও শেষ সারি জোড়া লাগান: "হিসাব ঠিক থাকলেও ভুল মানুষকে দিলে রিপোর্ট ভুল।"

২৩

পরের ক্লাস — "মাস শেষে টাকার হিসাব"

বিলিং ও প্ল্যাটফর্ম: `Subscription` · `Invoice._next_number` · read-only গেট · ট্রায়াল · সাসপেন্ড · প্ল্যাটফর্ম কনসোল

পর্ব ৪ · কোডিং · ক্লাস ২৩ / ৩৭ · পর্বের শেষ ক্লাস

মাস শেষে টাকার হিসাব

সিলেবাস: প্র্যাকটিক্যাল ৮ · টপিক ৮.১ (সাবস্ক্রিপশন ও বিলিং)

১৮টি বিলিং রুট · ৩৬টি টেস্ট · ২টি middleware · ৫টি সাবস্ক্রিপশন অবস্থা

is_writable · Invoice · read-only গেট

শুরু করুন প্রশ্ন দিয়ে

প্রতিষ্ঠান টাকা না দিলে ডেটা মুছে ফেলবেন?

হাত তুলুন: হ্যাঁ / না / ৩০ দিন পর

উত্তর: কিছুই মুছবে না — শুধু লেখা বন্ধ, পড়া খোলা।

স্কুলের ছাত্র-নম্বর-হাজিরা কারও বিল বাকির জিম্মায় নয়। এই একটি নৈতিক সিদ্ধান্ত থেকেই জন্মেছে `is_writable`, read-only গেট, আর "ডেটা নিরাপদ, শুধু নবায়ন করুন" বার্তা।

শিক্ষকের নোট: বলুন: "আপনার প্রোডাক্টের সবচেয়ে মানবিক কোডটি সাধারণত বিলিং ফাইলে থাকে।" এটা ক্লাসের সুর।

ধাপ ১

`Plan` — একটি দাম, একটি সারি

১৩টি ফিল্ড; সিড করা ৩টি প্ল্যান: starter · growth · campus

```
class Plan(models.Model):
    class Interval(models.TextChoices):
        MONTHLY = "MONTHLY", "Monthly"
        YEARLY = "YEARLY", "Yearly"

    code = models.SlugField(max_length=40, unique=True)
    name = models.CharField(max_length=80)
    price = models.DecimalField(max_digits=10,
                                decimal_places=2,
                                validators=[MinValueValidator(Decimal("0"))])
    interval = models.CharField(choices=Interval.choices)
    currency = models.CharField(max_length=3, default="BDT")
    max_students = models.PositiveIntegerField(null=True, blank=True)
    max_staff = models.PositiveIntegerField(null=True, blank=True)
    features = models.TextField(help_text="One feature per line.")
    trial_days = models.PositiveSmallIntegerField(default=14)
    is_public / is_active / sort_order

    def feature_list(self):
        return [l.strip() for l in self.features.splitlines() if l.strip()]
```

`null=True` মানে অসীম

`max_students=None` → "Unlimited students" (স্কুল প্ল্যান)।

টেক্সটে ফিচার

প্রতি লাইনে একটি — নতুন ফিচার যোগে মাইগ্রেশন লাগে না, শুধু লেখা বদলায়।

`is_active` বনাম `is_public`

পুরোনো গ্রাহক `is_active`-এ চলবে; নতুন বিক্রি বন্ধ হয় `is_public=False`-এ।

দাম `Decimal`

৳১৫০০.০০ — float নয়; ইনভয়েস আর রিপোর্টে একই সংখ্যা।

শিক্ষকের নোট: "প্ল্যান বদলালে পুরোনো গ্রাহকের দাম বদলে যাবে" — `Subscription`-এ `plan` FK রাখা হয়েছে, দাম কপি নয়; কিন্তু ইনভয়েসে দাম কপি হয় (পরে দেখা যাবে কেন)।

ধাপ ২

`Subscription` — প্রতিষ্ঠানপ্রতি একটি

`OneToOneField(Workspace)` · ৫টি অবস্থা · `seats_override`

```python
class Subscription(models.Model):
    class Status(models.TextChoices):
        TRIAL = "TRIAL", "Trial"
        ACTIVE = "ACTIVE", "Active"
        PAST_DUE = "PAST_DUE", "Payment overdue (read-only)"
        SUSPENDED = "SUSPENDED", "Suspended by platform"
        CANCELLED = "CANCELLED", "Cancelled"

    workspace = models.OneToOneField(Workspace,
        on_delete=models.CASCADE, related_name="subscription")
    plan = models.ForeignKey(Plan, on_delete=models.PROTECT)
    status = models.CharField(choices=Status.choices,
                              default=Status.TRIAL)
    trial_ends_on / current_period_start / current_period_end
    seats_override = models.PositiveIntegerField(null=True, blank=True)
```

সিদ্ধান্ত	কারণ
`OneToOne`	এক প্রতিষ্ঠানের দুই সাবস্ক্রিপশন মানে দ্বিগুণ বিল — অসম্ভব করে দেয় DB
`plan` -এ `PROTECT`	চালু গ্রাহকের প্ল্যান মুছে ফেলা যাবে না
`seats_override`	"৩০০ সিটের প্ল্যানে ৪০০" — দরকষাকষির জায়গা কোডে
পার্সোনাল ওয়ার্কস্পেস	কোনো সাবস্ক্রিপশনই নেই — `sub is None → writable`

> ব্যক্তিগত শিক্ষকের ওয়ার্কস্পেস কখনো গেটে আটকায় না — এটাও একটি টেস্টে লেখা (`test_personal_workspace_is_never_gated`)।

শিক্ষকের নোট: প্রশ্ন: "সাবস্ক্রিপশন কে বানায় — প্রতিষ্ঠান নাকি প্ল্যাটফর্ম?" উত্তর: `start_trial()` / `provision_institution()` — ব্যবহারকারী নিজে নয়।

ধাপ ৩

একটি property, পাঁচটি অবস্থা

`is_writable` — পুরো প্রোডাক্টের সবচেয়ে দামি ১০ লাইন



নিয়ম (কোডের ক্রমেই):

১. PAST_DUE · SUSPENDED · CANCELLED → False (সবসময়)
২. TRIAL → trial_ends_on খালি বা আজকের সমান/পরে হলে True
৩. ACTIVE → current_period_end একই নিয়ম · ৪. বাকি সব → False

মেয়াদোত্তীর্ণ trial = PAST_DUE-এর মতো:
ডেটা হারায় না, শুধু লেখা বন্ধ।
একটি property · সব ভিউ ও ব্যানার একই সত্য মানে

তারিখ আসে `timezone.localdate()` থেকে — সার্ভার UTC-তে থাকলেও স্কুলের "আজ" ঢাকার তারিখ।

শিক্ষকের নোট: বোর্ডে জিজ্ঞেস করুন: "TRIAL যার trial_ends_on=None — লেখা চালু না বন্ধ?" কোড বলে চালু (নিরাপদ দিক), কারণ `is None` চেক স্পষ্ট লেখা আছে।

ধাপ ৪

একটি তারিখ, চারটি প্রশ্ন

`ends_on` · `days_left` · `state_label` · `seat_percent`

```
@property
def ends_on(self):
    return (self.trial_ends_on if self.status == self.Status.TRIAL
            else self.current_period_end)

@property
def days_left(self):
    end = self.ends_on
    return None if end is None else (end - timezone.localdate()).days

@property
def state_label(self):
    if self.is_writable:
        return "Trial" if self.status == self.Status.TRIAL else "Active"
    return ("Expired" if self.status in (self.Status.TRIAL, self.Status.ACTIVE)
            else self.get_status_display())
```

প্রশ্ন	কোথায় উত্তর
"কবে শেষ?"	ব্যানার · ইনভয়েসের period_end
"কত দিন বাকি?"	`days_left` → "N দিন বাকি"
"কী অবস্থা?"	`state_label` — ব্যবহারকারীর ভাষায়
"সিট কতটা ভরা?"	`seat_percent` → ৯০% হলে সতর্কতা

`state_label`-এ "Expired" কৃত্রিম শব্দ — DB-তে নেই। ব্যবহারকারীকে "PAST_DUE" দেখানো যায় না, তাই অনুবাদ স্তর।

শিক্ষকের নোট: বলুন: "স্টেটাস কোড নয়, বাক্য দেখান।" `TRIAL` বনাম "ট্রায়াল চলছে, ৯ দিন বাকি" — ব্যবহারকারীর ভাষা আলাদা।

ধাপ ৫

গেট — লেখা আটকায়, পড়া নয়

`SubscriptionGateMiddleware` — সব ভিউয়ের আগে একবার

```
ALLOW_PREFIXES = ("/accounts/", "/renew/", "/platform/",
                  "/static/", "/media/", "/admin/", "/healthz",
                  "/workspaces/")
SAFE_METHODS = ("GET", "HEAD", "OPTIONS", "TRACE")

def __call__(self, request):
    if request.method in SAFE_METHODS:
        return self.get_response(request)
    workspace = self._workspace_in_url(request) or request.workspace
    if workspace is None or request.path.startswith(ALLOW_PREFIXES):
        return self.get_response(request)
    if is_writable(workspace):
        return self.get_response(request)
    # ORG_ADMIN → নবায়ন পর্দা; বাকিরা → হোম + বার্তা
    if membership.role == Roles.ORG_ADMIN:
        return redirect(reverse("billing:renew", kwargs={"slug": ...}) + "?blocked=1")
    messages.warning(request, "This institution's subscription has ended … "
                              "Ask your institution administrator to renew it — "
                              "your work is not lost.")
    return redirect(f"/w/{workspace.slug}/")
```

কেন middleware

ভিউ-তে লিখলে ৫৭টি ডেকোরেটরের সাথে আরেকটি জিনিস মনে রাখতে হতো।

GET কেন ছাড়

রিপোর্ট, হোমওয়ার্ক, মার্কশিট — সব পড়া যায়; ডেটা তাদেরই।

দুই ধরনের বার্তা

অ্যাডমিন নবায়ন পর্দায় (তিনিই ঠিক করতে পারেন), শিক্ষক হোমে — বিলিং তথ্য ফাঁস নয়।

বাস্তব বাগ

প্রথমে `request.workspace` (সেশন) দেখা হতো — এক শিক্ষক ব্যক্তিগত ওয়ার্কস্পেসে থাকা অবস্থায় মেয়াদোত্তীর্ণ স্কুলে POST করতে পারতেন। সমাধান: URL-এর slug।

শিক্ষকের নোট: বাগটা বোর্ডে আঁকুন: এক ব্যবহারকারী, দুই প্রতিষ্ঠান, সেশন একটায়, URL অন্যটায়। "নিরাপত্তার নিয়ম সবসময় URL-কে বিশ্বাস করবে, সেশনকে নয়।"

ধাপ ৬

গেটের সীমা — কী ছাড় পায়, কেন

ভুল ছাড় মানে ব্যবহারকারী আটকে যান; ভুল বন্ধ মানে গেট অর্থহীন

পথ	অবস্থা	কারণ
`/accounts/`	সবসময় খোলা	লগইন/লগআউট/পাসওয়ার্ড — নাহলে বেরোনোর পথই নেই
`/renew/`	সবসময় খোলা	সমস্যার সমাধান করতে হবে, বন্ধ করলে চিরকাল আটকে
`/platform/`	সবসময় খোলা	প্ল্যাটফর্ম অ্যাডমিন তিনিই বিল ঠিক করেন
`/media/` · `/static/`	সবসময় খোলা	ছবি/CSS বন্ধ হলে পর্দাই ভাঙে
`/w/<slug>/…`	লেখা বন্ধ	আসল গেট — উপস্থিতি, নম্বর, হোমওয়ার্ক

সুপারইউজার বাইপাস নেই — স্কুলের অ্যাডমিনও যদি প্ল্যাটফর্ম অ্যাডমিন হন, তিনিও মেয়াদ শেষে ওই স্কুলে লিখতে পারবেন না।

কনসোলের কাজ চলে `/platform/` ছাড়ে — তাই গেট নিজের পায়ে কুড়াল মারে না।

শিক্ষকের নোট: প্রশ্ন করুন: "ছাড়ের তালিকায় `/api/` থাকলে কী হবে?" — ভবিষ্যতে API যোগ করলে এই তালিকা আবার ভাবতে হবে; এটাই middleware-এর ঝুঁকি।

ধাপ ৭

`activate()` — টাকা, তারিখ, ইনভয়েস

এক ফাংশন, তিনটি লেখা, সবই `transaction.atomic`-এ

```python
@transaction.atomic
def activate(workspace, plan, *, days=None, amount=None,
             method=Invoice.Method.BKASH, reference="", actor=None):
    sub = ensure_subscription(workspace)
    today = timezone.localdate()
    start = today
    if (sub.current_period_end and sub.current_period_end > today
            and sub.plan_id == plan.pk):
        start = sub.current_period_end      # আগেভাগে নবায়ন: দিন হারায় না
    end = start + timedelta(days=days) if days else period_end(start, plan)

    sub.plan, sub.status = plan, Subscription.Status.ACTIVE
    sub.current_period_start, sub.current_period_end = start, end
    sub.save()

    return Invoice.objects.create(
        workspace=workspace, subscription=sub, plan=plan,
        period_start=start, period_end=end,
        amount=amount if amount is not None else plan.price,
        status=Invoice.Status.PAID, method=method,
        reference=reference[:80], paid_on=today, recorded_by=actor)
```

আগেভাগে নবায়ন

১৫ দিন বাকি থাকলে নতুন মেয়াদ শুরু হয় পুরোনোর শেষ থেকে — `test_early_renewal_extends_instead_of_resetting`।

ইনভয়েস = প্রমাণ

`status=PAID`, `paid_on`, `recorded_by` — কে, কখন, কত, কীভাবে।

দাম কপি হয়

প্ল্যানের দাম পরে বাড়লেও পুরোনো ইনভয়েস বদলায় না — আর্থিক নথি অপরিবর্তনীয়।

atomic কেন

সাবস্ক্রিপশন ACTIVE হলো কিন্তু ইনভয়েস লেখা ব্যর্থ — এমন অর্ধেক অবস্থা কখনো নয়।

শিক্ষকের নোট: "প্রমাণ ছাড়া টাকা নেওয়া বন্ধ করুন" — ইনভয়েস সংখ্যা, তারিখ আর রেফারেন্স তিনটিই থাকতে হবে; নাহলে পরে অভিযোগ মেটানো যায় না।

ধাপ ৮

ইনভয়েস নম্বর — ক্রম, এবং একটুখানি ঝুঁকি

`IMS-2026-0001` · `_next_number()` · `unique=True`

```python
def save(self, *args, **kwargs):
    if not self.number:
        self.number = self._next_number()
    super().save(*args, **kwargs)

@staticmethod
def _next_number() -> str:
    year = timezone.localdate().year
    prefix = f"{settings.INVOICE_PREFIX}-{year}-"
    last = (Invoice.objects.filter(number__startswith=prefix)
            .order_by("-number").values_list("number", flat=True).first())
    n = int(last.split("-")[-1]) + 1 if last else 1
    return f"{prefix}{n:04d}"
```

নীতি	কারণ
বছর prefix-এ	প্রতি বছর ০০০১ থেকে — হিসাবরক্ষকের প্রত্যাশা
`:04d`	সাজালে সংখ্যাগত ক্রম ঠিক থাকে (`-0009` < `-0010`)
`unique=True`	ডুপ্লিকেট নম্বর DB-ই আটকায়
শুধু খালি হলে তৈরি	সম্পাদনায় নম্বর বদলায় না

ঝুঁকি: দুইজন একসাথে ইনভয়েস বানালে দুজনেই একই "পরের নম্বর" পড়তে পারেন → একজন `IntegrityError` পান। সমাধান: `select_for_update()` বা ফাঁদে পড়ে রিট্রাই।

এখন ঝুঁকিটা ছোট কারণ ইনভয়েস বানান শুধু দুটি জায়গায় — তবু জানা থাকা দরকার।

শিক্ষকের নোট: দুটি টার্মিনাল খুলে একসাথে দুটি ইনভয়েস বানানোর চেষ্টা করুন — কখনো একটি ব্যর্থ হবে। "DB constraint হলো শেষ রক্ষা, প্রথম নয়।"

ধাপ ৯

সিট সীমা — সিদ্ধান্ত কোথায় নেওয়া হয়

`seat_block_reason()` → ৪র্থ ছাত্র যোগ করলে সার্ভারই না বলে

```
def seat_block_reason(workspace) -> str | None:
    sub = subscription_for(workspace)
    if sub is None:
        return None                 # ব্যক্তিগত ওয়ার্কস্পেস
    allowed = sub.seats_allowed     # seats_override or plan.max_students
    if allowed is None:
        return None                 # অসীম
    if sub.used_students() >= allowed:
        return (f"Your plan allows {allowed} active students. "
                f"Upgrade to add more.")
    return None

# ভিউতে (ছাত্র যোগের আগেই)
reason = seat_block_reason(request.workspace)
if reason:
    messages.error(request, reason)
    return redirect("students:list", slug=slug)
```

নিয়ম	প্রমাণ / কারণ
গণনা আসল সারি থেকে	`StudentProfile.objects.filter(status=ACTIVE).count()`
নিষ্ক্রিয় ছাত্র গোনায় নয়	পাশ করা ছাত্র সিট ছেড়ে দেয়
`seats_override` আগে	চুক্তি প্ল্যানের চেয়ে বড় হতে পারে
UI নয়, সার্ভার	`test_server_refuses_the_third_student`

> বার্তাটি কর্মের সাথে জোড়া: "Upgrade to add more" — শুধু "সীমা শেষ" নয়।

শিক্ষকের নোট: লাইভে সীমা ছুঁইয়ে দেখান: তৃতীয় ছাত্র যোগ করতে গেলে ভদ্র বার্তা; URL দিয়ে চেষ্টা করলেও একই। "নিয়ম UI-তে নয়, সার্ভারে।"

ধাপ ১০

কাস্টম ডোমেইন — প্রতিষ্ঠানের নিজের ঠিকানা

`CustomDomainMiddleware` · শুধু `domain_verified=True`

```python
class CustomDomainMiddleware:
    def __call__(self, request):
        request.domain_workspace = None
        host = request.get_host().split(":")[0].lower()
        branding = (WorkspaceBranding.objects
                    .select_related("workspace")
                    .filter(custom_domain__iexact=host,
                            domain_verified=True,
                            workspace__is_active=True)
                    .first())
        if branding is not None:
            request.domain_workspace = branding.workspace
            request.branding = branding
            if host not in settings.ALLOWED_HOSTS:
                settings.ALLOWED_HOSTS = list(settings.ALLOWED_HOSTS) + [host]
        response = self.get_response(request)
        patch_vary_headers(response, ["Host"])   # ক্যাশ আলাদা রাখে
        return response
```

ALLOWED_HOSTS কেন বদলায়

প্রোডাকশনে `*` নিষিদ্ধ — তাই যাচাই করা ডোমেইন রানটাইমে যোগ হয়, শুধু সেইগুলো।

চুরি করা যায় না

`test_domain_cannot_be_stolen_by_a_second_institution` — এক ডোমেইন এক প্রতিষ্ঠানের।

যাচাই ছাড়া কিছু নয়

`domain_verified=False` হলে উপেক্ষিত — `test_unverified_domain_is_ignored`।

Vary: Host

নাহলে CDN এক স্কুলের লগইন পর্দা আরেক স্কুলকে দেখাতে পারে।

শিক্ষকের নোট: বাস্তবে ডোমেইন যাচাই DNS TXT রেকর্ড দিয়ে হয় — কোডে শুধু `domain_verified` ফ্ল্যাগ; প্রক্রিয়াটা প্ল্যাটফর্ম অ্যাডমিনের হাতে।

ধাপ ১১

প্ল্যাটফর্ম কনসোল — টেন্যান্সির বাইরে

১৮টি রুট · শুধু সুপারইউজার · URL-এ কোনো `slug` নেই

```
/platform/                          dashboard
/platform/institutions/             তালিকা
/platform/institutions/new/         নতুন প্রতিষ্ঠান
/platform/institutions/<pk>/        বিস্তারিত
/platform/institutions/<pk>/activate/   পেমেন্ট + সক্রিয়
/platform/institutions/<pk>/toggle/     সক্রিয়/নিষ্ক্রিয়
/platform/institutions/<pk>/<action>/   suspend · past_due · …
/platform/plans/                    প্ল্যান তালিকা + নতুন
/platform/invoices/                 ইনভয়েস
/platform/signups/                  সাইনআপ অনুরোধ
```

নিয়ম	কারণ
URL-এ slug নেই	এটি কোনো প্রতিষ্ঠানের ভেতরের পর্দা নয়
`is_platform_admin`	`is_staff or is_superuser` — একটাই চেক
প্রতিষ্ঠান তৈরি	`provision_institution()` — ওয়ার্কস্পেস + অ্যাডমিন + ট্রায়াল, একসাথে
সাইনআপ অনুরোধ	ব্যর্থ হলেও অনুরোধ রাখা হয় (`SignupRequest`)

কনসোলে ভুল ক্লিক মানে ৪০০ স্কুলের বিল — তাই প্রতিটি কাজ `AuditLog`-এ লেখা হয় এবং কোনো ডিলিট বাটন নেই।

শিক্ষকের নোট: ডেমোতে `admin@ims.local` দিয়ে কনসোল দেখান: এক নতুন প্রতিষ্ঠান তৈরি করে ট্রায়াল শুরু করুন — পুরো প্রবাহ ৬০ সেকেন্ডে।

সারমর্ম

ছয়টি কথা সঙ্গে নিন

প্রতিটির পাশে IMS-এর প্রমাণ

নীতি	প্রমাণ
টাকা না দিলে ডেটা মুছবে না	`is_writable` · read-only · ৩৬টি টেস্ট
একটি property, সব পর্দায় একই সত্য	`ends_on` · `days_left` · `state_label`
গেট middleware-এ, ভিউয়ে নয়	`SubscriptionGateMiddleware` · `ALLOW_PREFIXES`
URL-এর স্কোপ, সেশনের নয়	`_workspace_in_url()` — বাস্তব বাগের সমাধান
টাকা আর নথি একসাথে	`activate()` · `transaction.atomic`
নম্বর অপরিবর্তনীয়	`IMS-2026-0001` · `unique` · `PROTECT`

শিক্ষকের নোট: প্রথম সারিটা আবার জোর দিন: "বিলিং কোড লেখার সময় আপনি ঠিক করেন আপনার প্রোডাক্ট কেমন মানুষ।"

পর্ব ৪ সমাপ্তি

আট ক্লাসে যা তৈরি হলো

১৫২ স্লাইড · ৩১টি মডেল · ৮৫টি রুট · ১০৩টি টেস্ট

ক্লাস	মূল ধারণা	দখল
১৬ · মডেল লেখা	২৩২ ফিল্ড · `clean()` · constraint	ডেটার আকৃতি
১৭ · ভিউ ও URL	`@workspace_view` · ৪০৩/৪০৪ · PRG	অনুরোধের পথ
১৮ · ফর্ম	হোয়াইটলিস্ট · স্কোপড queryset · `add_error`	ভুল ইনপুট
১৯ · টেমপ্লেট	৪টি শেল · partial · হোয়াইট-লেবেল	পর্দা
২০ · অথেন্টিকেশন	কাস্টম User · সেশন · রিসেট	পরিচয়
২১ · ফাইল আপলোড	`upload_to` · `clean()` · `FileResponse`	বাইনারি ডেটা
২২ · রিপোর্ট	`Decimal` · CSV BOM · মার্কশিট PDF	সিদ্ধান্ত
২৩ · বিলিং	`is_writable` · গেট · ইনভয়েস	টাকার নিয়ম

একই সুর আটবার: **নিয়ম সার্ভারে, প্রমাণ টেস্টে, বার্তা ব্যবহারকারীর ভাষায়** — এটাই পর্ব ৪-এর সারমর্ম।

শিক্ষকের নোট: এই টেবিলটা ছাত্রদের ডায়েরিতে লিখিয়ে নিন — পর্ব ৫ (টেস্টিং) শুরু হওয়ার আগে নিজেদের কোডে এই আটটি জায়গা খুঁজে দেখতে বলুন।

২৪

পর্ব ৫ শুরু — "যা পরীক্ষা করা হয়নি, তা নেই"

টেস্টিং: `TestCase` · `setUpTestData` · `force_login` · অনুমতির টেস্ট · টেস্ট পিরামিড · ১০৩টি টেস্ট থেকে শেখা